

**UNIVERSIDAD DE CUNDINAMARCA  
ESPECIALIZACIÓN EN INTELIGENCIA ARTIFICIAL  
PROCESAMIENTO DE LENGUAJE NATURAL**

**DOCENTE: PAUL ALEXANDER DÍAZ MONTAÑA**

**INFORME DE LABORATORIO: PROCESAMIENTO DE TEXTO**

**ESTUDIANTE: YOHAN NICOLÁS GÓMEZ CASTAÑEDA  
CC: 1233903906**

**FECHA: 27 DE SEPTIEMBRE DE 2026**

**REPOSITORIO:** <https://github.com/Nicolas2322/lab-procesamiento-texto>

## Contenido

1. Introducción
2. Marco teórico
  - 2.1 Del texto a las unidades léxicas
  - 2.2 Representación vectorial de documentos
  - 2.3 Análisis lingüístico
  - 2.4 Semántica distribucional
  - 2.5 Tareas de comprensión y transformación
  - 2.6 Generación de texto
  - 2.7 Minería de texto
3. Metodología
  - 3.1 Corpus de trabajo
  - 3.2 Conjuntos de evaluación anotados
  - 3.3 Organización del repositorio
  - 3.4 Entorno, reproducibilidad y manejo de datos
  - 3.5 Una restricción del entorno y su consecuencia metodológica
4. Resultados
  - 4.1 Normalización de texto
  - 4.2 Stopwords
  - 4.3 Stemming
  - 4.4 Lematización
  - 4.5 Term frequency
  - 4.6 Inverse document frequency
  - 4.7 Part-of-speech (tagging)
  - 4.8 Parsing
  - 4.9 Named-entity recognition
  - 4.10 Sentence similarity
  - 4.11 Text classification
  - 4.12 Question answering
  - 4.13 Summarization
  - 4.14 Translation
  - 4.15 Text generation
  - 4.16 Generación de texto usando LLM
  - 4.17 Text mining
5. Análisis y discusión
6. Limitaciones
7. Conclusiones
8. Referencias

## Procesamiento de texto

### 1. Introducción

Un modelo no lee palabras: lee números. Entre el texto tal como lo escribe un periodista y la matriz que recibe un algoritmo hay una cadena de decisiones —qué es una palabra, qué variantes cuentan como la misma unidad, qué se descarta por irrelevante— que suele tratarse como un paso previo y mecánico. Este laboratorio parte de la premisa contraria: cada una de esas decisiones elimina variación, y parte de la variación que elimina es ruido mientras que otra parte es información. La única manera de saber cuál es cuál es medirlo sobre el corpus concreto con el que se trabaja.

El laboratorio recorre las diecisiete técnicas de procesamiento de texto exigidas por la actividad, desde la normalización hasta la minería de texto, aplicándolas todas a un mismo corpus periodístico real en español: 998 noticias de la agencia EFE tomadas del corpus CoNLL-2002. Usar un único corpus a lo largo de todo el recorrido no es una comodidad, sino una condición del análisis: permite que los resultados de una técnica expliquen los de otra, que las decisiones de preprocesamiento se arrastren de forma consistente y que los errores observados en una tarea se rastreen hasta su origen en la anterior.

El criterio metodológico que organiza todo el trabajo es que ninguna técnica se presenta sola. Cada una se compara con una línea base sencilla y, siempre que es posible, se evalúa contra una anotación de referencia. El corpus elegido lo hace posible en varios temas, porque CoNLL-2002 incluye etiquetas gramaticales y de entidades nombradas asignadas por anotadores humanos; donde no existía referencia, se construyeron cinco conjuntos anotados a mano para este laboratorio, con un total de 191 elementos evaluados. El resultado es que las afirmaciones del informe se apoyan en cifras reproducibles y no en apreciaciones cualitativas.

El trabajo continúa el laboratorio anterior de la asignatura, *Transforma texto en embeddings*, que señalaba entre sus limitaciones el uso de un corpus sintético generado por plantillas. Este informe responde a esa limitación trabajando con texto real, con su ortografía imperfecta, sus abreviaturas y su ruido de dominio. También toma como punto de partida el material de clase alojado en el repositorio *dlaicourse* (Moroney, 2019), cuyo *Tokenizer* de Keras y cuyo modelo recurrente de predicción de la siguiente palabra se reproducen, se evalúan y, en un caso, se corrigen para el español.

## 2. Marco teórico

Las diecisiete técnicas del laboratorio no son una lista plana: forman una cadena en la que cada eslabón condiciona al siguiente. Esta sección las agrupa en siete bloques conceptuales que después estructuran los resultados.

### 2.1 Del texto a las unidades léxicas

La **normalización** transforma el texto para que las formas que deben contar como una misma unidad se escriban igual. Incluye la tokenización —la separación en unidades mínimas—, el paso a minúsculas, el tratamiento de signos, números y ruido de dominio. No existe una normalización correcta en abstracto: cada operación destruye una distinción, y la pregunta pertinente es si esa distinción importaba para la tarea (Jurafsky y Martin, 2025).

Las **stopwords** o palabras vacías son términos muy frecuentes y de función gramatical —artículos, preposiciones, conjunciones, auxiliares— que se eliminan porque discriminan poco entre documentos. Su eliminación reduce drásticamente el número de tokens, pero la definición de «palabra vacía» depende de la tarea y las listas estándar son decisiones que otros tomaron para otros fines.

El **stemming** reduce cada palabra a una raíz eliminando sufijos mediante reglas, sin diccionario ni contexto; el algoritmo Snowball para el español aplica en orden la eliminación de pronombres enclíticos, de sufijos derivativos, de sufijos verbales y de vocales finales (Porter, 2001). La **lematización** persigue el mismo objetivo —agrupar las variantes de una palabra— pero devuelve el lema, la forma de entrada de diccionario, y para ello necesita conocer la categoría gramatical del token, es decir, su contexto. La diferencia entre ambas no es de grado sino de naturaleza: una opera sobre las letras finales y la otra sobre el análisis morfológico.

### 2.2 Representación vectorial de documentos

El modelo de espacio vectorial representa cada documento como un vector cuyas componentes son términos. La **frecuencia de término** (*term frequency*, TF) mide cuántas veces aparece un término en un documento y responde a la intuición de que un término repetido describe mejor el documento; su versión cruda, sin embargo, favorece sistemáticamente a los documentos largos, por lo que se emplean variantes: binaria, relativa, sublineal logarítmica y normalizada (Salton y Buckley, 1988).

La **frecuencia inversa de documento** (*inverse document frequency*, IDF) corrige el problema complementario: un término que aparece en todos los documentos no distingue ninguno. Spärck Jones (1972) propuso ponderar cada término por el inverso del número de documentos en que aparece, de modo que el peso crezca con la rareza. El producto  $TF \cdot IDF$  combina ambas señales y sigue siendo, medio siglo después, la línea base contra la que se

comparan las representaciones densas. BM25 (Robertson y Zaragoza, 2009) refina la idea introduciendo saturación de la frecuencia y normalización de longitud parametrizables.

### 2.3 Análisis lingüístico

El **etiquetado gramatical** (*part-of-speech tagging*) asigna a cada token su categoría —sustantivo, verbo, determinante— resolviendo la ambigüedad mediante el contexto. Es la entrada de casi todo lo demás: la lematización depende de él, y también la extracción de sintagmas. El **análisis sintáctico** (*parsing*) establece la estructura de la oración; la gramática de dependencias, hoy dominante, representa esa estructura como relaciones binarias entre una palabra núcleo y sus dependientes, lo que resulta más robusto y más portable entre lenguas que los árboles de constituyentes (Nivre et al., 2020).

El **reconocimiento de entidades nombradas** (*named-entity recognition*) identifica y clasifica los tramos de texto que designan personas, organizaciones, lugares y otras categorías. La tarea compartida CoNLL-2002 (Tjong Kim Sang, 2002) fijó para el español el esquema de anotación IOB y las cuatro categorías que este laboratorio utiliza como referencia.

### 2.4 Semántica distribucional

La hipótesis distribucional sostiene que las palabras que aparecen en contextos parecidos tienen significados parecidos. De ahí derivan las representaciones densas: el análisis semántico latente descompone la matriz término-documento para descubrir dimensiones latentes (Deerwester et al., 1990), y Word2Vec aprende vectores prediciendo el contexto de cada palabra (Mikolov et al., 2013). La **similitud de oraciones** mide el parecido semántico entre dos textos; componer vectores de palabra en vectores de oración por promedio simple funciona sorprendentemente bien, y ponderar ese promedio por la rareza de cada palabra lo mejora (Arora et al., 2017), aunque ninguna composición por promedio captura el orden.

La **clasificación de texto** asigna a cada documento una etiqueta de un conjunto cerrado. Es la tarea supervisada canónica del área y sirve aquí como evaluación extrínseca: si una representación captura el contenido de un documento, un clasificador sencillo entrenado sobre ella debe funcionar bien incluso con pocos ejemplos.

### 2.5 Tareas de comprensión y transformación

Las **preguntas y respuestas** (*question answering*) en su variante extractiva consisten en devolver el tramo literal de un documento que responde a una pregunta. El conjunto SQuAD 2.0 (Rajpurkar et al., 2018) añadió preguntas sin respuesta para forzar a los sistemas a abstenerse, y fijó las métricas hoy estándar: coincidencia exacta y F1 de solapamiento de palabras.

El **resumen automático** puede ser extractivo —seleccionar oraciones del documento— o abstractivo —generar texto nuevo—. Entre los métodos extractivos, TextRank construye un grafo cuyos nodos son oraciones y cuyas aristas son similitudes, y aplica PageRank para encontrar las más centrales (Mihalcea y Tarau, 2004); LexRank emplea centralidad con umbral sobre el mismo tipo de grafo (Erkan y Radev, 2004). En noticias de agencia la línea base *Lead*, que se limita a tomar las primeras oraciones, es notoriamente difícil de superar por la estructura de pirámide invertida del género.

La **traducción automática** pasó de sistemas basados en reglas y diccionarios a modelos estadísticos y, finalmente, neuronales. Sus métricas de evaluación automática comparan la salida con traducciones de referencia: BLEU mide coincidencia de n-gramas de palabras (Papineni et al., 2002) y chrF lo hace sobre n-gramas de caracteres, lo que la vuelve más tolerante con la morfología y más fiable en muestras pequeñas (Popović, 2015).

## 2.6 Generación de texto

Generar texto es muestrear repetidamente la siguiente palabra de una distribución condicionada por las anteriores. Los modelos de **n-gramas** estiman esa distribución contando: son interpretables y rápidos, pero sufren dispersión de datos, ya que la mayoría de los contextos largos nunca se observan. Los modelos neuronales recurrentes sustituyen el conteo por una representación distribuida que generaliza a contextos no vistos. La estrategia de muestreo importa tanto como el modelo: la búsqueda voraz degenera en repeticiones, mientras que la temperatura y el muestreo top-k permiten graduar el compromiso entre coherencia y diversidad (Holtzman et al., 2020).

Los **grandes modelos de lenguaje** (LLM) llevan esa misma idea a una escala en la que emerge un comportamiento distinto: en lugar de entrenarse para cada tarea, se les describe la tarea en el propio texto de entrada. Brown et al. (2020) mostraron que basta con instrucciones (*zero-shot*) o con unos pocos ejemplos en el contexto (*few-shot*) para obtener resultados competitivos sin actualizar un solo parámetro. Esto cambia la unidad de trabajo del ingeniero: del entrenamiento al diseño de instrucciones.

## 2.7 Minería de texto

La minería de texto busca patrones en colecciones grandes sin partir de etiquetas. Las **colocaciones** son combinaciones de palabras que aparecen juntas más de lo que predice el azar; la información mutua puntual mide esa asociación, con el sesgo conocido de premiar coocurrencias raras, mientras que la razón de verosimilitud resulta más estable en frecuencias bajas (Church y Hanks, 1990). El **modelado de temas** mediante asignación latente de Dirichlet (Blei et al., 2003) representa cada documento como una mezcla de temas y cada tema como una distribución sobre palabras, lo que permite descubrir la estructura temática de una colección sin supervisión.

## 3. Metodología

### 3.1 Corpus de trabajo

Se emplea la parte en español del corpus CoNLL-2002 (Tjong Kim Sang, 2002), compuesto por despachos de la agencia EFE publicados en mayo de 2000 y distribuido con NLTK. La elección responde a tres razones. Primera, es texto real, con la ortografía, las abreviaturas y el ruido propios de la prensa. Segunda, incorpora anotación humana de categoría gramatical, con el juego de etiquetas EAGLES, y de entidades nombradas en formato IOB, lo que convierte en evaluables varios de los temas del laboratorio. Tercera, se descarga de forma programática, de modo que todos los notebooks son reproducibles sin adjuntar datos.

El corpus se distribuye tokenizado y segmentado en oraciones, sin marcas de documento. Para recuperar las noticias completas se implementó una reconstrucción que detecta la cabecera característica de los despachos —«Ciudad, fecha (EFE)»— y reagrupa las oraciones, además de revertir la tokenización para obtener texto corrido. El resultado son 998 noticias, 11.755 oraciones y 369.171 tokens, con una mediana de 313 tokens por noticia.

### 3.2 Conjuntos de evaluación anotados

El corpus no incluye preguntas, temas, pares de similitud ni traducciones. Para poder evaluar esos temas con números y no con impresiones, se construyeron cinco conjuntos anotados a mano, versionados en la carpeta datos/ del repositorio.

**Tabla 1***Conjuntos de evaluación contruidos para el laboratorio*

| Archivo              | Contenido                                                                                                                           | Elementos |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------|
| lemas_referencia.csv | Palabras en contexto con su lema correcto, agrupadas por fenómeno (flexión, derivación, verbos irregulares, enclíticos, homografía) | 42        |
| sts_es.csv           | Pares de oraciones con similitud de 0 a 5 y su fenómeno (paráfrasis, brecha léxica, negación, roles invertidos, polisemia...)       | 26        |
| temas_eфе.csv        | Noticias del corpus etiquetadas en cinco temas equilibrados                                                                         | 70        |
| qa_eфе.csv           | Preguntas sobre ocho noticias, con respuestas copiadas literalmente del texto; cuatro no tienen respuesta en la noticia             | 37        |
| traduccion_es_en.csv | Oraciones con traducción de referencia al inglés: de noticias, con entidades y cifras, y modismos                                   | 16        |

*Nota.* Son conjuntos pequeños: sirven para comparar métodos y analizar errores concretos, no para estimar con precisión el rendimiento absoluto. Por eso cada sección de resultados examina los aciertos y los fallos uno a uno además de las métricas agregadas.

### 3.3 Organización del repositorio

Todo el código está en un único repositorio público, organizado en tres capas. La capa de biblioteca, `src/pln_lab/`, reúne once módulos con el código reutilizable: carga del corpus, preprocesamiento, ponderación, etiquetado, similitud, preguntas y respuestas, resumen, traducción, generación y evaluación. La capa de datos, `datos/`, contiene los conjuntos anotados. La capa de experimentación, `notebooks/`, contiene seis notebooks que únicamente orquestan: ninguno duplica lógica de otro, y sus salidas —figuras y métricas— se guardan en subcarpetas.

El agrupamiento de los diecisiete temas en seis notebooks sigue la afinidad técnica y no el orden del enunciado, de modo que cada notebook mantiene un hilo argumental propio y comparte el preprocesamiento entre temas vecinos.



**Tabla 2***Correspondencia entre los temas exigidos y los notebooks del repositorio*

| <b>Notebook</b>                                        | <b>Temas del laboratorio</b>                                   |
|--------------------------------------------------------|----------------------------------------------------------------|
| 01 · Normalización, stopwords, stemming y lematización | Normalización de texto · Stopwords · Stemming · Lematización   |
| 02 · Term frequency e inverse document frequency       | Term frequency · Inverse document frequency                    |
| 03 · Etiquetado, análisis sintáctico y entidades       | Part-of-speech (tagging) · Parsing · Named-entity recognition  |
| 04 · Similitud de oraciones y clasificación            | Sentence similarity · Text classification                      |
| 05 · Preguntas y respuestas, resumen y traducción      | Question answering · Summarization · Translation               |
| 06 · Generación de texto y minería de texto            | Text generation · Generación de texto usando LLM · Text mining |

### 3.4 Entorno, reproducibilidad y manejo de datos

Los notebooks se ejecutaron con Python 3.11 sobre CPU, con spaCy 3.8 y el modelo es\_core\_news\_md, NLTK 3.10, scikit-learn 1.8, gensim 4.4 y TensorFlow 2.20. Son autocontenidos: la primera celda detecta si se está ejecutando en Google Colab y, en ese caso, clona el repositorio e instala el modelo de spaCy, de manera que cualquiera puede abrirlos desde los enlaces del archivo README.md y reejecutarlos sin configuración previa.

El manejo de datos sigue tres reglas. Los datos de partida se obtienen de forma programática y nunca se editan a mano. Los conjuntos anotados se versionan junto al código, porque son parte del experimento. Los productos derivados pesados quedan fuera del repositorio y se regeneran ejecutando los notebooks. A esto se añade el control de la aleatoriedad —semilla fija en todos los notebooks, y entrenamiento de Word2Vec con un solo hilo porque gensim no es determinista con varios— y la trazabilidad de las cifras: cada notebook vuelca sus métricas a un archivo JSON, de modo que todo número citado en este informe puede rastrearse hasta la celda que lo produjo.

### 3.5 Una restricción del entorno y su consecuencia metodológica

El entorno de ejecución empleado no tiene acceso de red al repositorio de modelos de Hugging Face. Esta restricción afecta a los temas que en la práctica profesional se resolverían hoy con modelos Transformer preentrenados: similitud de oraciones, preguntas y respuestas, resumen abstractivo, traducción y generación con grandes modelos.

La decisión metodológica adoptada fue construir, para cada uno de esos temas, un sistema propio con las herramientas disponibles —recuperación con TF-IDF y BM25, extracción guiada por el análisis gramatical, TextRank, traducción por diccionario y reglas, modelos de n-gramas y

una red recurrente entrenada desde cero— y evaluarlo con las métricas estándar de cada tarea. El objetivo no es simular un Transformer, sino disponer de una línea base medida cuyos errores permitan razonar con precisión sobre qué aporta exactamente la arquitectura moderna. Un sistema clásico bien evaluado dice más sobre el problema que un modelo preentrenado usado como caja negra. En cada sección afectada se indica qué resolvería un Transformer y por qué, y los notebooks incluyen el código correspondiente, comentado y listo para ejecutarse en Colab con GPU.

## 4. Resultados

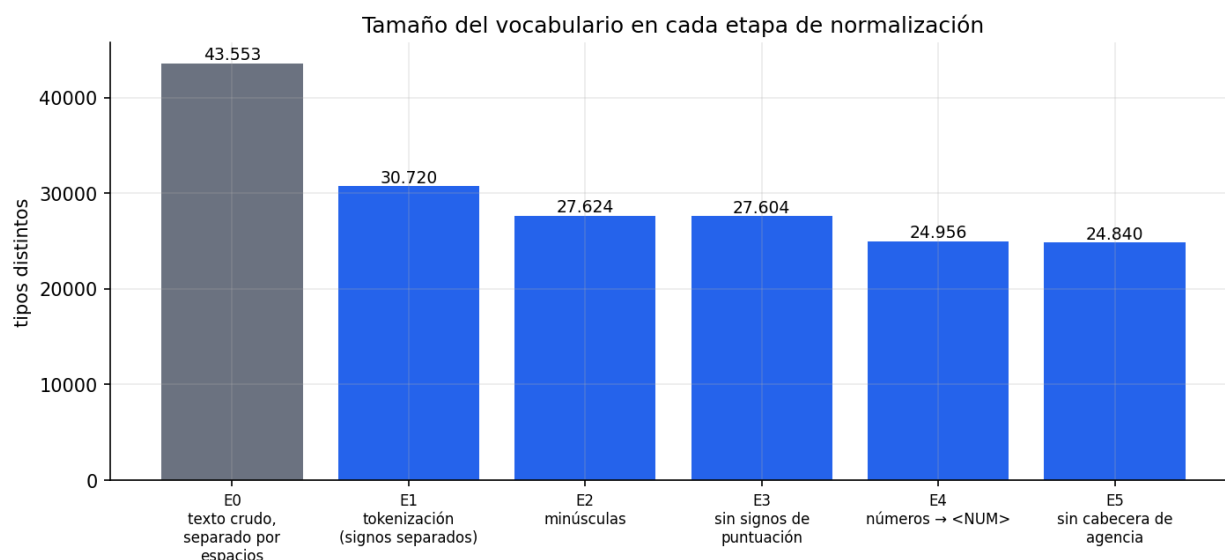
Los resultados se presentan en el orden de los temas exigidos por el laboratorio. Cada apartado indica el notebook del que procede, describe el experimento, presenta las cifras obtenidas e interpreta lo que revelan.

### 4.1 Normalización de texto

La normalización se midió etapa por etapa sobre las 998 noticias, observando el tamaño del vocabulario —número de formas distintas— después de cada operación. El resultado más contraintuitivo es que la operación de mayor efecto no es ninguna de las que suelen asociarse con «normalizar», sino la **tokenización**: separar los signos de puntuación reduce el vocabulario un 29,5 %, de 43.553 a 30.720 formas, porque al separar solo por espacios *Gobierno*, *Gobierno*, y *Gobierno*. son tres unidades distintas. El paso a minúsculas aporta un 10,1 % adicional y la sustitución de números por un marcador un 9,6 %; en conjunto, la cadena reduce el vocabulario un 43,0 % sin haber tocado todavía la morfología.

**Figura 1**

*Tamaño del vocabulario en cada etapa de la cadena de normalización*



**Análisis:** el descenso se concentra en la primera etapa. La eliminación de los signos de puntuación apenas modifica el número de formas distintas —los signos son pocos aunque muy frecuentes en tokens—, mientras que la última etapa, la supresión de la cabecera de agencia, casi no altera el vocabulario pero sí las frecuencias: reduce las apariciones de «efe» de 1.418 a 409 y las de «may» de 1.035 a 71, que de otro modo figurarían entre las palabras más frecuentes del corpus por un artefacto del formato de los despachos.

Se reprodujo además el *Tokenizer* de Keras utilizado en el laboratorio de clase (Moroney, 2019) y se comprobó que sus filtros por defecto, diseñados para el inglés, **no incluyen los signos de apertura del español** (¿, ¡, «, »). En consecuencia, ¿Qué y qué quedan como dos entradas distintas del diccionario. En este corpus el efecto es menor —15 formas afectadas, porque la prensa formula pocas preguntas directas—, pero en reseñas o redes sociales duplicaría parte del vocabulario. Es un error silencioso: el código funciona y el resultado parece razonable.

Dos decisiones de la cadena se tomaron tras medir su costo. La primera es conservar las mayúsculas en las etapas de análisis lingüístico: la anotación de entidades del corpus permitió comprobar que 613 formas funden una entidad con una palabra común al pasar a minúsculas —*Gobierno* con *gobierno*, *Unión* con *unión*—, y que esas formas concentran el 21,6 % de los tokens que integran entidades nombradas. La segunda es **conservar las tildes**. Eliminarlas corrige errores tipográficos reales (*pais*, *futbol*, *comunicacion*: 462 grupos que suman 13.259 tokens), pero borra distinciones gramaticales en 59 grupos que suman 57.704 tokens, porque afectan justamente a las palabras más frecuentes del idioma: *el* frente a *él*, *esta* frente a *está*, *si*

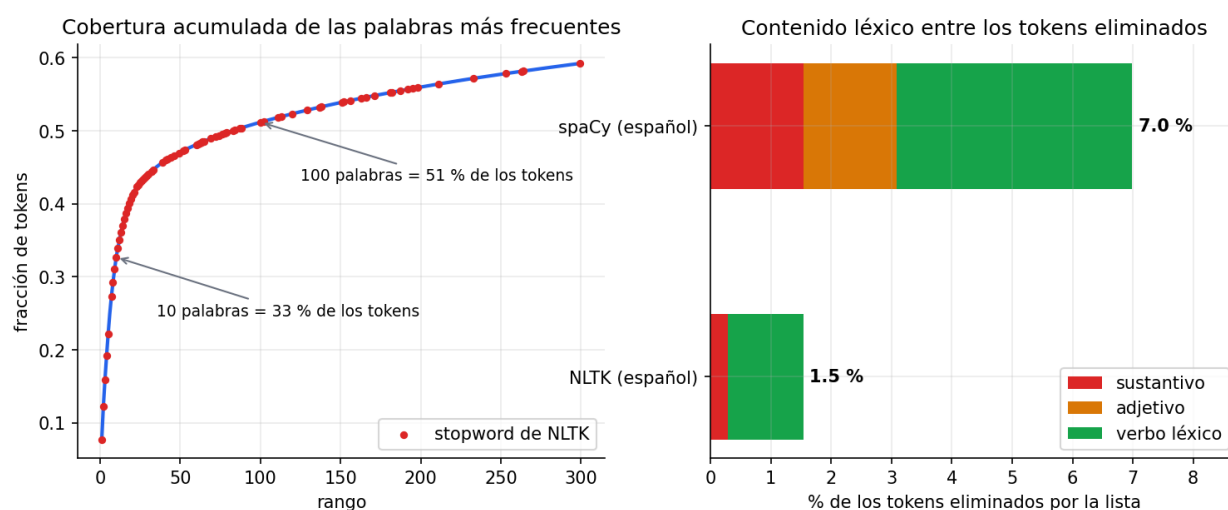
frente a *sí*, *mas* frente a *más*. La relación es de más de cuatro tokens dañados por cada token corregido.

## 4.2 Stopwords

Se compararon tres listas de palabras vacías: la del laboratorio de clase —en inglés, procedente del ejercicio con noticias de la BBC—, la de NLTK para español y la de spaCy para español. Aplicada al corpus, la lista en inglés elimina solo el 2,0 % de los tokens, y lo hace por **coincidencias accidentales** entre idiomas: *a* es aquí una preposición española, *he* una forma del verbo haber y, de forma más dañina, *once* —«una vez» en inglés— es el número *once* en español, que desaparecería de frases como «once personas murieron». Reutilizar el código del laboratorio sin cambiar la lista produce un preprocesamiento que aparenta funcionar y casi no hace nada.

**Figura 2**

*Cobertura de las palabras más frecuentes y contenido léxico eliminado por cada lista*



**Análisis:** el panel izquierdo muestra por qué las stopwords son tan rentables: diez palabras concentran el 33 % de los tokens del corpus y cien alcanzan el 51 %, y casi todas ellas pertenecen a la lista de NLTK. El panel derecho muestra el precio: de lo que elimina NLTK, solo el 1,5 % son sustantivos, adjetivos o verbos léxicos, mientras que en spaCy esa proporción se cuadruplica hasta el 7,0 %, con términos como «acuerdo», «parte», «próximo», «dijo» o «explicó», que en prensa son contenido informativo y no palabras vacías.

La auditoría se realizó cruzando cada token eliminado con su etiqueta gramatical de referencia, lo que permite afirmar con datos —y no por inspección— qué destruye cada lista. El hallazgo con mayor consecuencia práctica es que **ambas listas incluyen negaciones** (*no*, *ni*,

*nada, sin*; spaCy añade *nunca, nadie, tampoco*), de modo que «el acuerdo fue aprobado» y «el acuerdo no fue aprobado» producen exactamente la misma representación. Para clasificación temática la pérdida es inocua; para análisis de sentimiento o detección de contradicciones invierte el significado. Se adoptó por ello la lista de NLTK sin negaciones, que elimina el 45,0 % de los tokens, y la eliminación de stopwords se restringió a las tareas de bolsa de palabras.

### 4.3 Stemming

El algoritmo Snowball para español reduce el vocabulario de contenido de 24.636 a 13.060 formas, un 47,0 %, y divide por más de dos el número de *hapax* —formas que aparecen una sola vez—, de 10.565 a 4.710. En términos prácticos eso significa matrices documento-término más pequeñas y menos dispersas. Las familias más grandes muestran su virtud: más de veinte formas de *presentar* o de *celebrar*, incluidos tiempos verbales, participios y derivados, colapsan en una sola unidad.

Las mismas familias muestran sus dos defectos característicos, que tienen una causa común: el algoritmo no sabe qué palabra está procesando, solo qué letras tiene al final.

**Tabla 3**

*Los dos tipos de error del stemming, con ejemplos del corpus*

| Error            | Qué ocurre                                             | Ejemplos                                                                                                   |
|------------------|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Sobre-agrupación | Palabras de significado distinto reciben la misma raíz | <i>crear / creer → cre; parte / partido / partir → part; operación / ópera → oper; autoridades → autor</i> |
| Sub-agrupación   | Una misma palabra queda dispersa en varias raíces      | <i>hacer → hac, hiz, hech, hag, hic; decir → dij, dec, dic, dich; año / años quedan separados</i>          |

*Nota.* La sub-agrupación afecta sobre todo a los verbos irregulares, que son los más frecuentes del idioma, por lo que su peso en tokens es muy superior al que sugiere el número de casos.

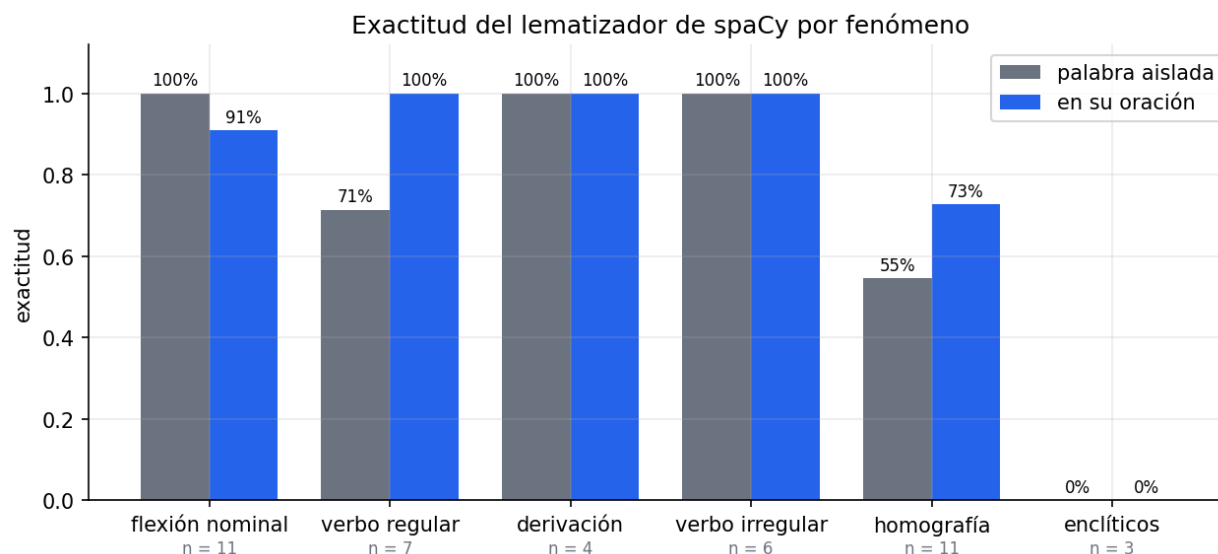
El caso de *parte, partido y partir* ilustra el costo real: una noticia sobre un partido político y otra sobre la partida de un tren quedan, tras el stemming, más cerca de lo que están. Y el caso de *año* frente a *años* muestra que ni siquiera la flexión regular está garantizada, porque el algoritmo no elimina la -s final después de ñ.

### 4.4 Lematización

La lematización se evaluó con el conjunto de 42 casos construido para el laboratorio, cada uno con una palabra objetivo dentro de una oración y su lema correcto. Cada palabra se lematizó de dos maneras: dentro de su oración y aislada. El lematizador de spaCy acierta el **83,3 %** con contexto y el **76,2 %** sin él, y la diferencia no se reparte por igual entre fenómenos.

**Figura 3**

*Exactitud del lematizador por fenómeno lingüístico, con y sin contexto*



**Análisis:** la flexión nominal, la derivación y los verbos irregulares se resuelven prácticamente siempre, incluso sin contexto, gracias a las tablas de excepciones: donde el stemming fallaba («autoridades» → «autor», «peces» → «pec»), el lematizador devuelve «autoridad» y «pez». El contexto se vuelve decisivo en dos fenómenos. En los verbos regulares, los participios aislados se interpretan como adjetivos («publicados» → «publicado») y solo dentro de la oración se reconocen como verbo («publicar»). En la homografía, la exactitud sube de 54,5 % a 72,7 %: aislada, «vino» siempre es el verbo «venir». Los enclíticos son el punto ciego absoluto, con 0 % en ambas condiciones.

Sobre el corpus completo, los enclíticos generan 777 tokens con lema compuesto (*hacerlo* → «hacer él», *presentarse* → «presentar él»), que fragmentan el vocabulario porque *presentar* y «presentar él» cuentan como lemas distintos; y 565 tokens reciben un lema inexistente (*reúne* → «reúnir», *acudan* → «acudar»), donde el modelo acierta que se trata de un verbo pero aplica la regla de una conjugación equivocada. Son pocos frente a los 164.491 tokens de contenido, pero indican que en producción conviene separar el enclítico antes de lematizar o validar el lema contra un diccionario.

La comparación directa entre ambas técnicas, sobre los mismos tokens, resume el compromiso.

**Tabla 4***Stemming frente a lematización sobre los mismos tokens de contenido*

| <b>Criterio</b>                          | <b>Stemming (Snowball)</b>                 | <b>Lematización (spaCy)</b>                              |
|------------------------------------------|--------------------------------------------|----------------------------------------------------------|
| Vocabulario resultante                   | 12.992 formas (-47,1 %)                    | 17.670 formas (-28,0 %)                                  |
| Tokens cuya forma es una palabra real    | 68,7 %                                     | 94,8 %                                                   |
| Tokens cuya forma agrupa lemas distintos | 65,8 %                                     | —                                                        |
| Uso del contexto                         | Ninguno: <i>vino</i> siempre es <i>vin</i> | Sí, con límites ( <i>fue</i> casi siempre → <i>ser</i> ) |
| Costo                                    | 3,4 veces más rápido, sin modelo           | Requiere un modelo neuronal (≈ 40 MB)                    |

*Nota.* La referencia para la penúltima fila son las propias palabras del corpus, que alcanzan un 95,5 % de formas reales con el mismo criterio.

El stemming reduce más porque **agrupa de más**: el 65,8 % de los tokens de contenido tiene una raíz que reúne dos o más lemas distintos. Esa cifra incluye agrupaciones razonables para un buscador (*trabajo / trabajar*) junto con fusiones erróneas, pero demuestra que la reducción adicional no es gratuita. El criterio de elección que se adoptó es funcional: el stemming conviene cuando importa la velocidad y basta con agrupar de forma aproximada —indexación para búsqueda, conteos masivos—; la lematización, cuando los resultados deben ser legibles o cuando mezclar palabras distintas es costoso. Para los notebooks de representación y minería se adoptó la lematización.

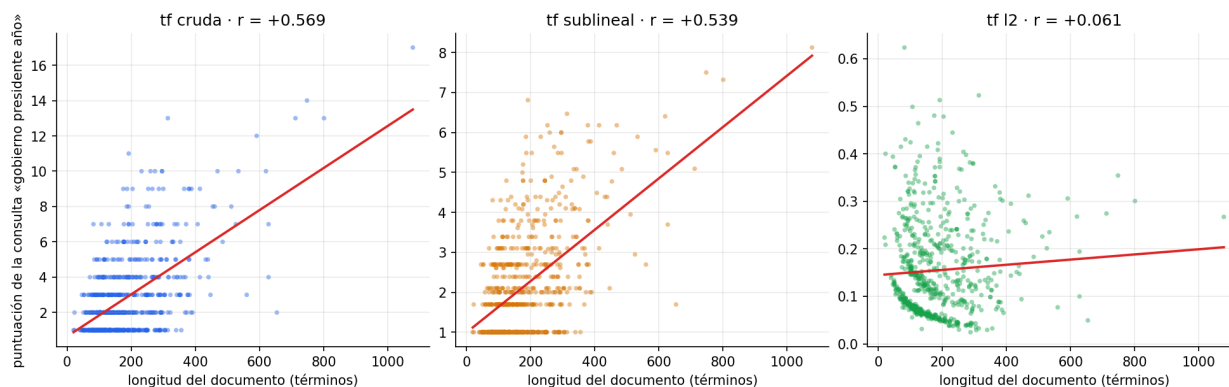
#### 4.5 Term frequency

Sobre el corpus lematizado y sin stopwords se construyó la matriz documento-término, de  $998 \times 17.357$  y con un **99,31 %** de celdas nulas: solo 119.053 de los más de 17 millones de valores posibles son distintos de cero. Esa dispersión extrema es la característica que define a las representaciones de bolsa de palabras y la razón de que se almacenen en matrices dispersas.

Se compararon cinco variantes de ponderación —cruda, binaria, relativa, sublineal y normalizada L2— y se cuantificó el sesgo de longitud midiendo la correlación entre la longitud del documento y la puntuación que recibe para una consulta dada.

**Figura 4**

*Sesgo de longitud de las variantes de ponderación de la frecuencia de término*



**Análisis:** la frecuencia cruda correlaciona con la longitud del documento en  $r = +0,569$ : un documento no obtiene una puntuación alta por hablar del tema, sino por ser largo. Las diez noticias mejor puntuadas con esta variante tienen una longitud mediana de 530 términos frente a los 138 del corpus. La normalización L2 reduce esa correlación a  $+0,061$ , y la frecuencia relativa la invierte ( $-0,228$ ), sobreponderando los documentos muy cortos. La ponderación sublineal, que aplica logaritmo a la frecuencia, corrige la saturación pero no el sesgo de longitud ( $+0,539$ ): son dos problemas distintos que exigen dos correcciones distintas.

Esta distinción entre saturación y longitud se confirmó de forma independiente en la evaluación de recuperación del apartado siguiente: al pasar de la frecuencia cruda con producto punto a la misma frecuencia con similitud coseno —es decir, únicamente normalizando la longitud— la precisión@5 sube de 0,660 a 0,786, una ganancia de 0,126 puntos. Añadir IDF sobre esa base aporta 0,037 puntos adicionales. **La normalización de longitud rinde más que el propio IDF**, un resultado que contradice el orden habitual de exposición del tema.

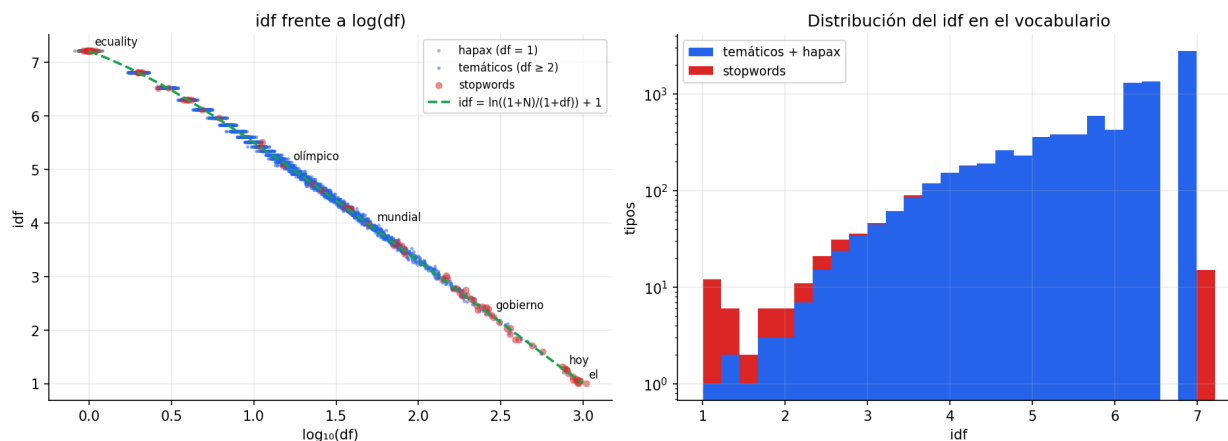
#### 4.6 Inverse document frequency

El IDF se calculó a mano con la fórmula suavizada que emplea scikit-learn,  $\ln((1+N)/(1+df)) + 1$ , y se verificó contra el atributo `idf_` de `TfidfVectorizer`: la diferencia máxima absoluta sobre los 17.357 términos es **0,0**. La verificación no es un trámite, sino la manera de asegurar que la implementación manual y la de la biblioteca comparten exactamente la misma definición, incluido el suavizado, que varía entre textos y bibliotecas.



**Figura 5**

*Relación entre frecuencia documental e IDF, con los tres grupos de términos*



**Análisis:** la relación es logarítmica por construcción, pero lo informativo es dónde cae cada grupo. Las stopwords ocupan el 0,5 % del vocabulario y el 44,8 % de los tokens, con un IDF medio de 4,00 que baja a 1,28 en las veinte más frecuentes. En el extremo opuesto, los hapax son el 48,0 % del vocabulario y solo el 3,4 % de los tokens, con el IDF máximo de 7,21. El IDF, por tanto, no hace más que reordenar el vocabulario según una escala que la propia distribución de Zipf ya anticipaba en el notebook 01, donde la pendiente log-log medida fue de  $-0,949$  frente al  $-1$  teórico.

El efecto del IDF sobre la descripción de los documentos es sustancial: **cambia el término más representativo en 518 de las 998 noticias (51,9 %)**, y el solapamiento medio entre los diez primeros términos con TF y con TF-IDF es de 6,46 sobre 10. En una noticia sobre comercio electrónico, por ejemplo, *millón* cae del puesto 4 al 16 y *año* del 11 al 73, mientras que un término específico del documento asciende del 10 al 4.

La utilidad del IDF se midió de forma extrínseca con la tarea de recuperar noticias del mismo tema sobre las 70 noticias etiquetadas, donde el azar produce 0,188.

**Tabla 5***Precisión@5 en recuperación temática según el esquema de ponderación*

| Esquema de ponderación                                    | Precisión@5 |
|-----------------------------------------------------------|-------------|
| Azar                                                      | 0,188       |
| Frecuencia cruda, producto punto                          | 0,660       |
| Frecuencia cruda, coseno (solo normalización de longitud) | 0,786       |
| TF-IDF                                                    | 0,823       |
| BM25 ( <i>b</i> óptimo)                                   | 0,831       |
| TF-IDF sublineal                                          | 0,849       |

*Nota.* Precisión@5 es la fracción de las cinco noticias más similares que comparten el tema de la noticia consultada, promediada sobre las 70 noticias etiquetadas.

El recorte del vocabulario mediante `min_df` y `max_df` confirma la asimetría de la distribución: exigir que un término aparezca en al menos dos documentos elimina el 48,2 % del vocabulario conservando el 93,5 % de los tokens, mientras que descartar los términos presentes en más del 10 % de los documentos elimina apenas 156 términos pero un 24 % de los tokens. La primera operación abarata el modelo casi sin perder información; la segunda cambia radicalmente lo que el modelo puede ver.

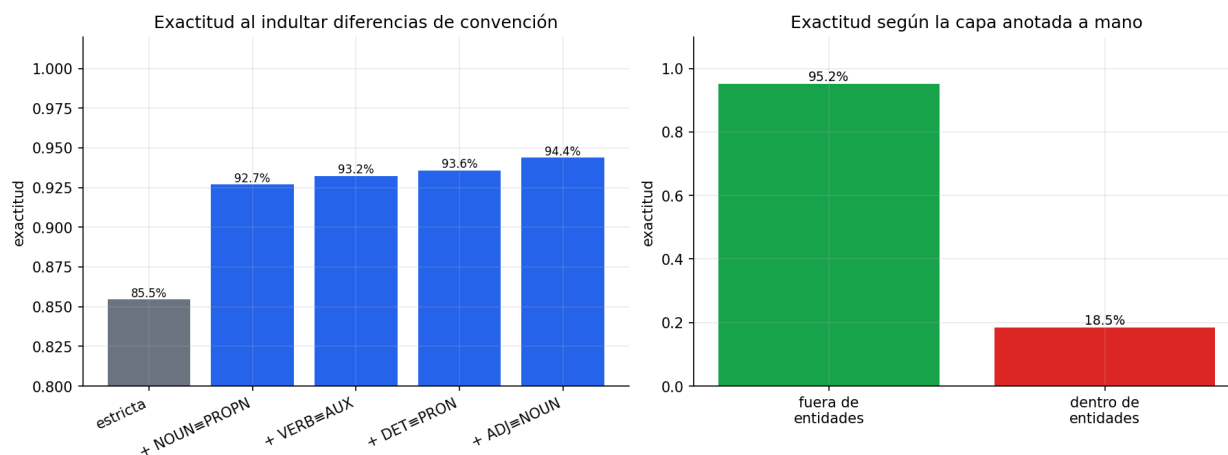
#### 4.7 Part-of-speech (tagging)

El etiquetado gramatical se evaluó sobre una muestra de 3.000 oraciones y 107.033 tokens, comparando spaCy con la anotación humana del corpus. Como ambos usan juegos de etiquetas distintos —EAGLES en el corpus, universal en spaCy— se construyó y documentó un mapeo explícito de las 60 etiquetas EAGLES presentes a las 14 categorías universales. La exactitud global resultante es del **85,5 %**, una cifra que a primera vista parece baja para una herramienta madura.

El análisis de los desacuerdos revela que la cifra no mide lo que aparenta. Separando los tokens según formen parte o no de una entidad nombrada, la exactitud es del **95,2 % fuera de las entidades** y del **18,5 % dentro**: el 71 % de todos los desacuerdos se concentra en el 13 % de tokens que pertenecen a entidades. La causa es una diferencia de criterio de anotación, no un error del etiquetador: CoNLL-2002 etiqueta la mayoría de los nombres propios como sustantivos comunes —27.142 formas con mayúscula anotadas como NC frente a solo 1.846 como NP—, mientras que spaCy los reconoce como nombres propios. Tratando ambas categorías como equivalentes, la exactitud asciende al 92,7 %.

**Figura 6**

*Exactitud del etiquetado dentro y fuera de las entidades nombradas*

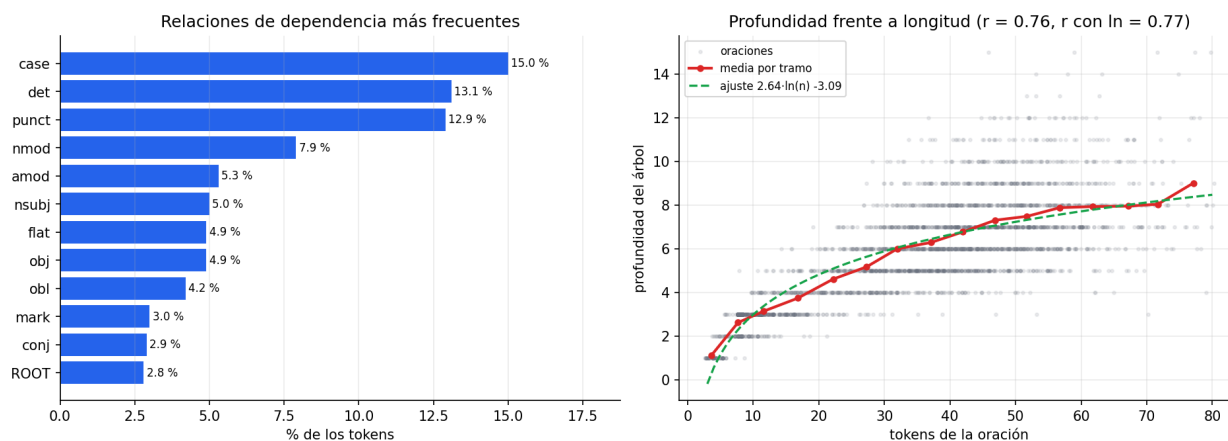


**Análisis:** la separación entre ambos regímenes es la conclusión metodológica del apartado: una métrica agregada puede ocultar por completo la naturaleza del error. Sin esta descomposición se habría concluido que spaCy etiqueta mal uno de cada siete tokens del español, cuando en realidad su desempeño sobre el texto ordinario es del 95,2 % y lo que se está midiendo en el resto es la discrepancia entre dos convenciones de anotación separadas por veinte años.

La comparación con una línea base es igual de instructiva. Un etiquetador que se limita a asignar a cada palabra su etiqueta más frecuente en el propio corpus alcanza un 92,4 % con el criterio estricto, es decir, **supera a spaCy**, precisamente porque aprende la convención de anotación del corpus, incluida la de los nombres propios. Pero su ventaja desaparece donde importa: en las palabras que no vio durante el entrenamiento —el 4,5 % de la muestra— cae al 32,3 % frente al 65,6 % de spaCy. La línea base memoriza; el modelo generaliza. Es un recordatorio de que una línea base competitiva obliga a precisar qué se está midiendo, y que un número aislado nunca decide entre dos sistemas.

#### 4.8 Parsing

El análisis sintáctico de dependencias se realizó con spaCy sobre el mismo corpus. La profundidad media de los árboles es de 5,85 niveles, y su relación con la longitud de la oración se ajusta mejor a una función logarítmica ( $2,64 \cdot \ln n - 3,09$ , con  $r = 0,775$ ) que a una lineal: las oraciones largas no se vuelven proporcionalmente más profundas, sino que se ensanchan mediante coordinación y adjunción. Dos relaciones, case y det, concentran el 28 % de los tokens, lo que refleja el peso de preposiciones y determinantes en la sintaxis del español.

**Figura 7***Estructura de los árboles de dependencias del corpus*

**Análisis:** la extracción de tripletas sujeto-verbo-objeto, que es el uso más habitual del análisis sintáctico en minería de información, solo tiene éxito en el 15,9 % de los verbos del corpus. La razón es lingüística y no técnica: en español el sujeto se omite con frecuencia, abundan las pasivas reflejas y las construcciones con verbo de dicción («informó a EFE que...»), y muchos complementos son preposicionales en lugar de objetos directos. Quien diseñe una extracción de hechos basada en tripletas sobre texto en español debe contar con que perderá la mayor parte de los enunciados.

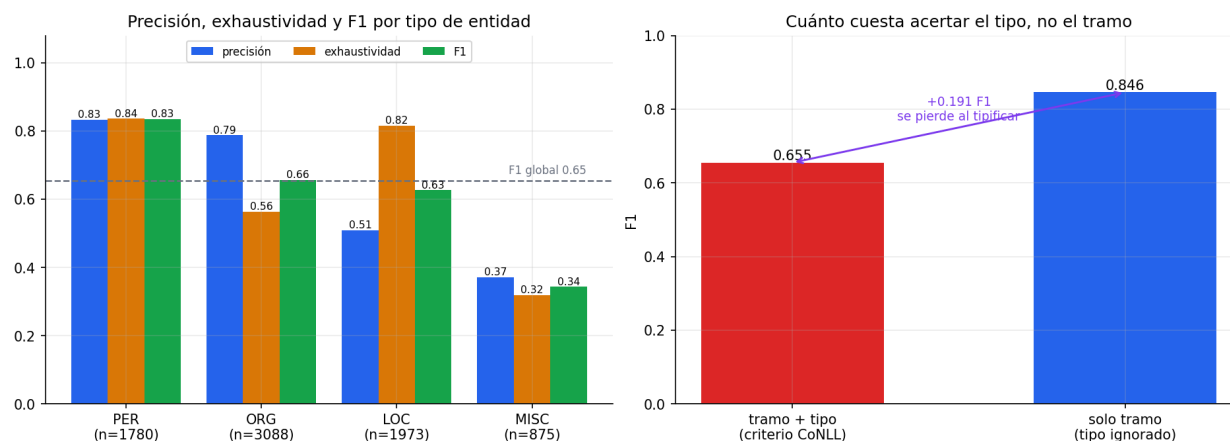
Se examinó también un caso de ambigüedad de adjunción de sintagma preposicional, el fenómeno clásico de «vi al hombre con el telescopio» que el propio docente empleó como ejemplo en clase. En el par mínimo analizado, el sintagma «con antecedentes penales» se adjunta al verbo en lugar de al sustantivo, y la adjunción de «en el Congreso» cambia de núcleo al alargar la oración. El analizador no dispone de conocimiento del mundo para decidir, de modo que resuelve por regularidades estadísticas de su entrenamiento; es exactamente el tipo de decisión que los modelos contextuales modernos aciertan con más frecuencia, pero tampoco de manera garantizada.

#### 4.9 Named-entity recognition

El reconocimiento de entidades es el tema donde la anotación de referencia del corpus resulta más valiosa, porque permite una evaluación estricta: se cuenta un acierto solo cuando coinciden a la vez el tramo exacto y el tipo de entidad. La implementación de la extracción de tramos IOB y del emparejamiento se hizo de forma propia para controlar el criterio. Sobre 7.716 entidades anotadas, spaCy obtiene una **F1 global de 0,655** (precisión 0,647, exhaustividad 0,663).

**Figura 8**

*Precisión, exhaustividad y F1 del reconocimiento de entidades por tipo*



**Análisis:** el rendimiento varía mucho entre categorías: las personas se reconocen con F1 de 0,835, las organizaciones con 0,657 y los lugares con 0,627, mientras que la categoría MISC se desploma a 0,343. MISC agrupa entidades heterogéneas —gentilicios, nombres de acontecimientos, competiciones, premios— cuya definición depende de decisiones de anotación específicas del corpus, y ningún modelo entrenado en otra colección puede adivinarlas.

La descomposición de los errores señala con precisión dónde está la dificultad. De los desaciertos, 1.491 son de tipo, 841 de frontera, 267 son entidades no detectadas y 517 entidades espurias. Relajando el criterio para exigir únicamente que el tramo coincida, sin importar el tipo, la F1 sube de 0,655 a **0,846**: el modelo *encuentra* las entidades y falla sobre todo al *clasificarlas*. La confusión dominante es organización etiquetada como lugar, con 626 casos, y tiene una explicación lingüística clara: en el lenguaje periodístico los topónimos funcionan por metonimia como instituciones o equipos —«Madrid negocia con Bruselas», «el Valencia perdió»—, y esa distinción depende del contexto de la oración, no de la forma del nombre.

Este resultado tiene una consecuencia directa sobre la cadena de procesamiento: dado que el 21,6 % de los tokens de entidad se funde con palabras comunes al pasar a minúsculas (apartado 4.1), el reconocimiento de entidades debe ejecutarse **antes** de la normalización agresiva, y no después. El orden de las operaciones, que suele tratarse como un detalle de implementación, determina aquí el techo de rendimiento alcanzable.

#### 4.10 Sentence similarity

La similitud semántica entre oraciones se evaluó con el conjunto de 26 pares anotados, comparando cinco representaciones: TF-IDF, análisis semántico latente, promedio de los vectores estáticos de spaCy, promedio de vectores Word2Vec entrenados sobre el propio corpus y ese mismo promedio ponderado con el esquema SIF. La métrica principal es la correlación de Spearman con la anotación humana, porque interesa si el método *ordena* los pares como lo hace una persona y no si reproduce su escala.

**Tabla 6**

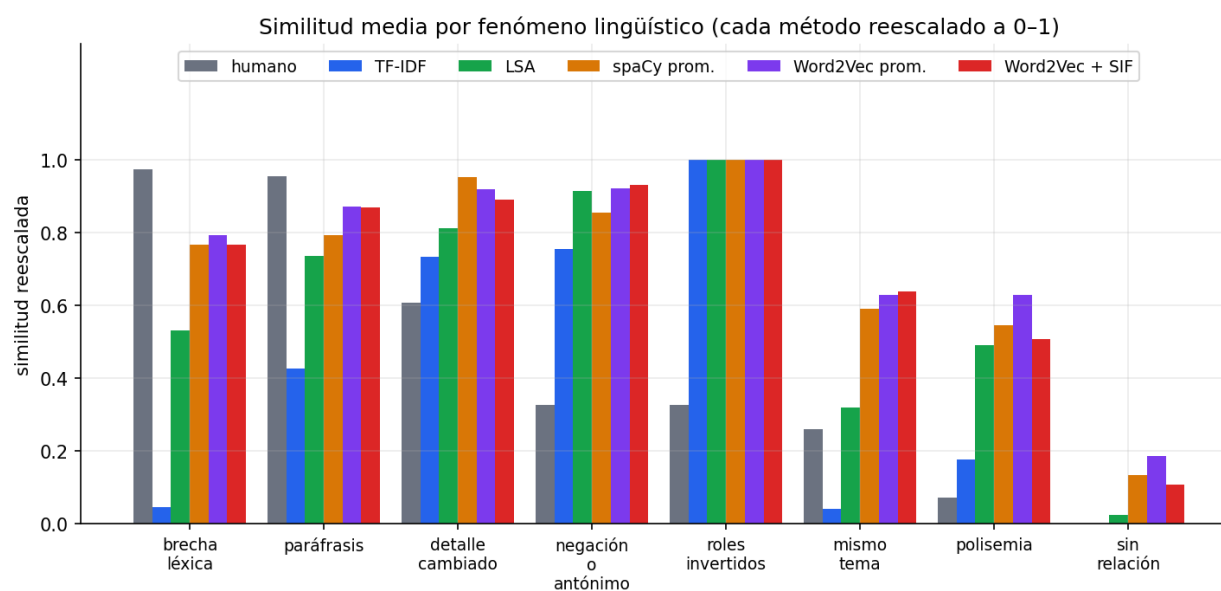
*Correlación con la anotación humana y precisión@5 en recuperación*

| Representación                        | Spearman | Pearson | Precisión@5 |
|---------------------------------------|----------|---------|-------------|
| TF-IDF                                | 0,207    | -0,009  | 0,851       |
| Análisis semántico latente (150 dim.) | 0,358    | 0,325   | 0,849       |
| Word2Vec, promedio                    | 0,446    | 0,490   | 0,931       |
| Word2Vec + SIF                        | 0,473    | 0,518   | 0,963       |
| spaCy, promedio de vectores           | 0,549    | 0,507   | 0,889       |

*Nota.* La correlación de TF-IDF no alcanza significación estadística ( $p = 0,31$ ). El azar en precisión@5 es 0,188.

**Figura 9**

*Similitud media por fenómeno lingüístico: anotación humana frente a cada representación*



**Análisis:** el perfil por fenómeno explica lo que la correlación global resume. En la brecha léxica —pares que significan lo mismo sin compartir palabras— TF-IDF da 0,045 donde la anotación humana da 0,974, y los embeddings alcanzan entre 0,77 y 0,79: es la victoria clara de las representaciones densas y confirma el hallazgo del laboratorio anterior. Pero en los tres fenómenos críticos todos los métodos fracasan por igual: en roles invertidos («el Real Madrid venció al Valencia» frente a «el Valencia venció al Real Madrid») los cinco dan exactamente 1,00 frente al 0,33 humano, y en negación oscilan entre 0,76 y 0,93 frente al mismo 0,33.

La razón de ese fracaso es estructural y no un defecto de entrenamiento: todas las representaciones evaluadas son composiciones por promedio, y un promedio es invariante ante el orden de sus términos. Dos oraciones con las mismas palabras en distinto orden tienen literalmente el mismo vector, de modo que la similitud debe ser 1,00 por construcción. La negación falla por una razón emparentada: *no* aporta un único vector entre muchos, y su efecto semántico —invertir el valor de verdad de toda la oración— no es una operación que el promedio pueda representar. Aquí es donde un codificador basado en Transformer marca la diferencia, porque construye la representación de cada token en función de los demás y puede, en principio, distinguir el sujeto del objeto.

En la aplicación práctica de recuperación, los embeddings vuelven a imponerse: Word2Vec con ponderación SIF alcanza 0,963 de precisión@5 frente a 0,851 de TF-IDF. Un ejemplo ilustra la diferencia cualitativa: ante una consulta sobre fútbol, TF-IDF devuelve como primer resultado una noticia sobre el «Jamón de Teruel» —que menciona la palabra *liga* en otro sentido—, mientras que el modelo latente devuelve tres noticias deportivas.

#### 4.11 Text classification

La clasificación temática se evaluó sobre las 70 noticias etiquetadas en cinco categorías, con validación cruzada estratificada de cinco particiones. Se compararon seis modelos que abarcan las dos familias: representaciones dispersas con clasificadores lineales y representaciones densas, más una red neuronal construida al estilo del laboratorio de clase.

**Tabla 7***Exactitud de los modelos de clasificación temática (validación cruzada de cinco particiones)*

| <b>Modelo</b>                                                      | <b>Exactitud</b> |
|--------------------------------------------------------------------|------------------|
| Word2Vec promediado + regresión logística                          | 1,000            |
| TF-IDF + máquina de vectores de soporte lineal                     | 0,957            |
| TF-IDF + regresión logística                                       | 0,957            |
| Análisis semántico latente + regresión logística                   | 0,957            |
| Red de Keras ( <i>Embedding + GlobalAveragePooling1D + Dense</i> ) | 0,957            |
| spaCy promediado + regresión logística                             | 0,943            |
| TF-IDF + Naive Bayes multinomial                                   | 0,914            |

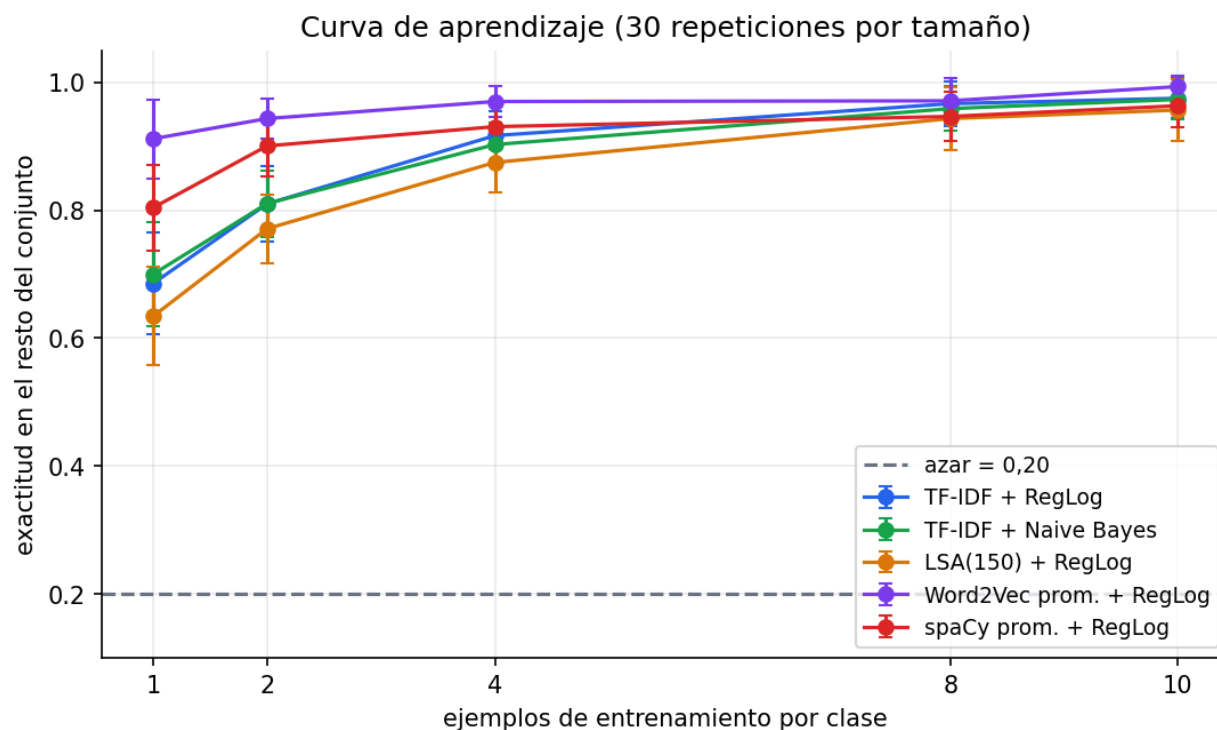
*Nota.* La red de Keras reproduce la arquitectura del laboratorio de clase (Moroney, 2019) y entrena 100.181 parámetros.

El resultado más útil no es cuál gana, sino que **casi todos empatan** en torno al 95 %. Con cinco temas muy distintos y noticias largas, la tarea está saturada: las palabras que separan deportes de economía son tantas y tan evidentes que cualquier clasificador lineal razonable las encuentra. La red neuronal, con cien mil parámetros, no mejora a una regresión logística sobre TF-IDF, lo que confirma que la complejidad del modelo no compensa cuando el cuello de botella está en los datos.



**Figura 10**

*Exactitud según el número de noticias etiquetadas por tema*



**Análisis:** la diferencia real entre representaciones solo aparece en el régimen de pocos datos. Con un único ejemplo por tema, Word2Vec promediado alcanza 0,912 frente al 0,686 de TF-IDF, una ventaja de 22,6 puntos; con diez ejemplos la brecha se reduce a 1,8 puntos. La explicación es la brecha léxica del apartado anterior: con un solo ejemplo, TF-IDF solo puede clasificar correctamente las noticias que repiten las palabras de ese ejemplo, mientras que el espacio denso sitúa cerca a las que hablan de lo mismo con otras palabras. Es el argumento cuantitativo a favor de las representaciones densas cuando etiquetar es caro.

Los tres errores del modelo léxico sobre las setenta noticias son informativos por sí mismos: la ruptura sentimental de un tenista se clasifica como cultura, la caída de un ciclista como sucesos y unas inundaciones en Colombia como deportes. En los tres casos el vocabulario de superficie apunta a la categoría equivocada y solo el conocimiento del dominio —que una lesión deportiva sigue siendo deporte— permitiría acertar.

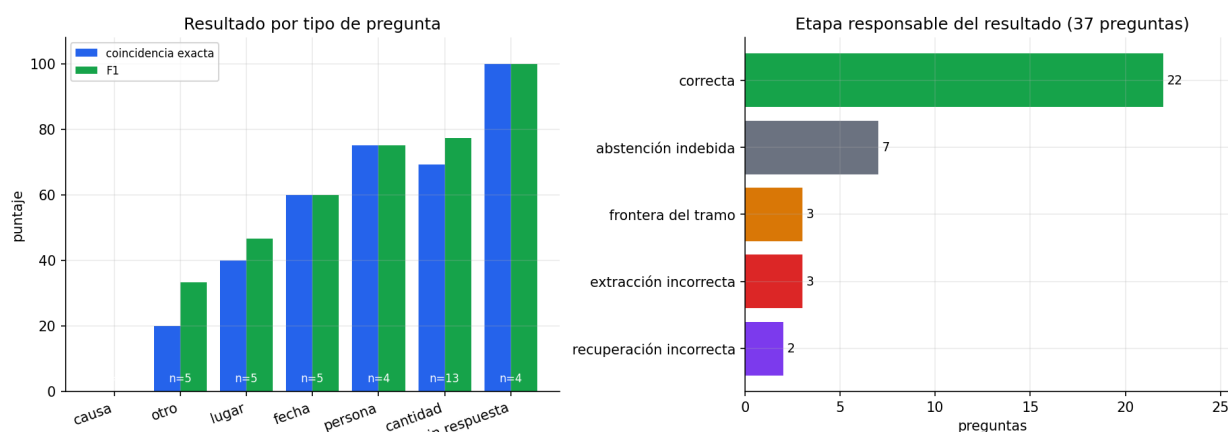
## 4.12 Question answering

Se construyó un sistema de preguntas y respuestas extractivo propio, en tres etapas: recuperación de la oración candidata mediante TF-IDF o BM25, extracción del tramo respuesta con reglas guiadas por el tipo de pregunta y apoyadas en el análisis gramatical y de entidades de spaCy, y un umbral de confianza para abstenerse. Se evaluó con las métricas de SQuAD sobre las 37 preguntas anotadas, cuatro de las cuales no tienen respuesta en la noticia. El resultado global es de **59,5 de coincidencia exacta** y **65,0 de F1**, con las cuatro abstenciones correctamente detectadas.

La descomposición por etapas localiza el cuello de botella con precisión. La recuperación acierta la oración correcta en el 81,8 % de los casos en la primera posición, en el 97,0 % entre las tres primeras y en el **100 % entre las cinco primeras**. Es decir, la información siempre se encuentra: lo que falla es la extracción del tramo exacto dentro de la oración.

**Figura 11**

*F1 del sistema de preguntas y respuestas según el tipo de pregunta*



**Análisis:** el rendimiento se ordena exactamente según lo que las reglas pueden aprovechar. Las preguntas por persona (75,0 de coincidencia exacta) y por cantidad (69,2, con F1 de 77,4) funcionan porque el reconocedor de entidades y el etiquetador identifican directamente el tipo de respuesta buscado. Las de lugar bajan a 40,0, las abiertas de tipo «qué» a 20,0 y las de causa a 0,0. Una pregunta causal («¿por qué el avión no pudo regresar?») no tiene un tipo de entidad asociado: la respuesta es un sintagma verbal cuya delimitación exige comprender la relación causal expresada en la oración, y ninguna regla superficial la recorta correctamente.

El análisis de los 37 casos arroja 22 respuestas correctas, 7 abstenciones indebidas, 3 errores de frontera del tramo, 3 de extracción y 2 de recuperación. Ese perfil de errores describe con exactitud lo que aporta un modelo extractivo tipo BERT ajustado en SQuAD: no una mejor

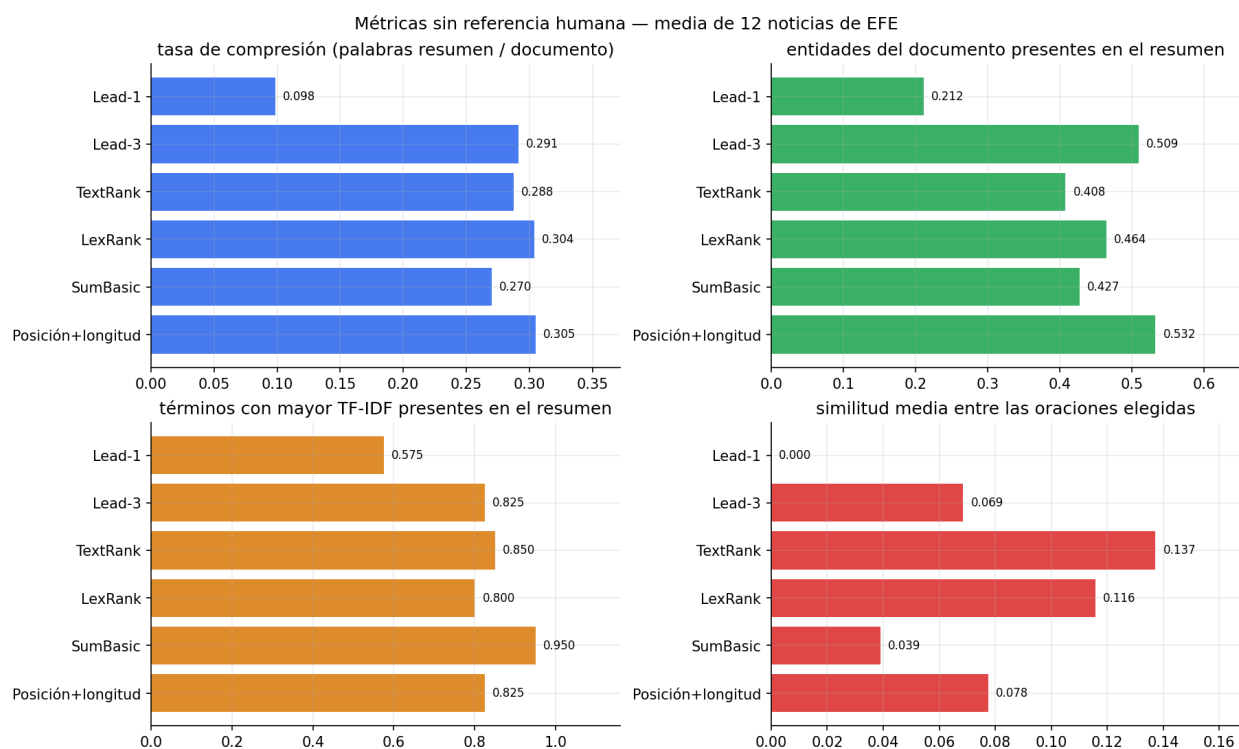
recuperación —la recuperación ya es perfecta a cinco oraciones— sino la sustitución de las reglas de extracción por dos distribuciones aprendidas sobre el inicio y el final de la respuesta, que delimitan tramos arbitrarios sin necesidad de que exista una categoría de entidad. Los notebooks incluyen ese código comentado para ejecutarlo en un entorno con acceso a los modelos.

### 4.13 Summarization

Se compararon cuatro métodos de resumen extractivo sobre doce noticias: las líneas base Lead-1 y Lead-3, que toman las primeras oraciones; TextRank, implementado de forma propia sobre un grafo de oraciones ponderado por similitud TF-IDF; LexRank; y una selección por posición y longitud. Al no existir resúmenes de referencia humanos, la evaluación se apoyó en métricas sin referencia: compresión, cobertura de las entidades nombradas del documento, cobertura de los términos con mayor TF-IDF y redundancia entre las oraciones seleccionadas.

**Figura 12**

*Métricas sin referencia de los métodos de resumen extractivo*



**Análisis:** Lead-3 conserva más entidades del documento que TextRank (0,509 frente a 0,408) y produce menos redundancia entre oraciones (0,069 frente a 0,137), mientras que SumBasic domina en cobertura de términos temáticos (0,950 frente a 0,825). Los tres criterios apuntan en direcciones distintas, lo que evidencia que no existe una única noción de «buen resumen» y que

la métrica elegida decide el ganador.

El hallazgo central del apartado es que **Lead-3 es una línea base extraordinariamente difícil de superar en este género**. La explicación no es algorítmica sino editorial: las noticias de agencia se redactan con la estructura de pirámide invertida, de modo que el primer párrafo contiene deliberadamente el qué, quién, cuándo y dónde. Un método que optimiza centralidad en un grafo compite contra un texto que ya fue resumido por un profesional en su primera oración. De hecho, TextRank termina seleccionando la entradilla en 11 de las 12 noticias, es decir, converge hacia Lead sin saberlo, y cuando se aparta de ella suele elegir oraciones centrales por vocabulario pero periféricas por información, como declaraciones secundarias.

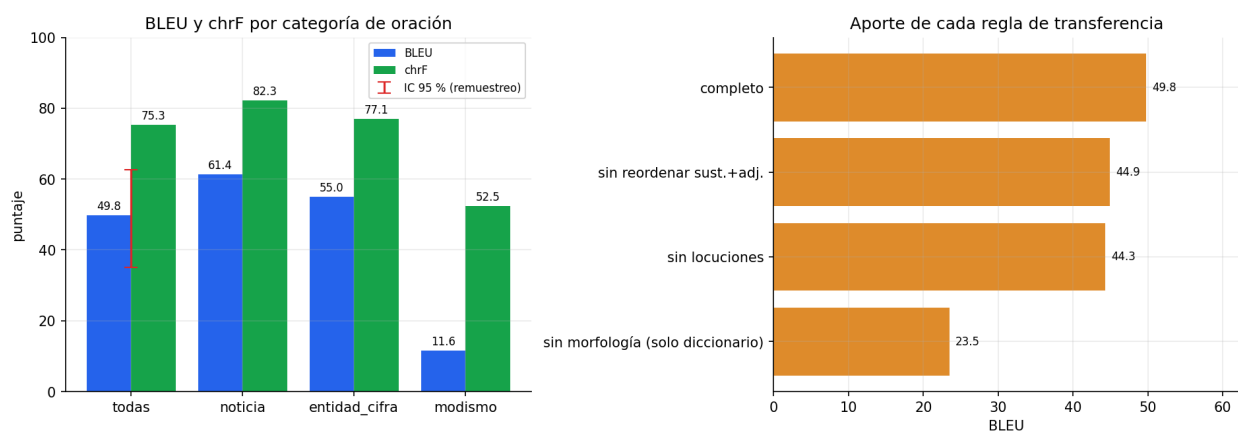
La comparación ROUGE tomando Lead-3 como pseudo-referencia —una referencia aproximada, no humana, y así se declara— sitúa la selección por posición y longitud en 0,951 de ROUGE-1, TextRank en 0,661 y LexRank en 0,611, lo que cuantifica esa convergencia. La limitación de fondo de todo el apartado es que el resumen extractivo no puede fundir dos hechos dispersos en una sola frase ni reformular: es precisamente lo que aportaría un modelo abstractivo, con el riesgo añadido de inventar datos que no están en el documento.

#### 4.14 Translation

Se implementó un traductor español-inglés basado en diccionario y reglas —330 entradas léxicas, 15 locuciones, reordenamiento de sustantivo y adjetivo, concordancia y tratamiento de artículos— y se evaluó con BLEU y chrF sobre las 16 oraciones de referencia construidas para el laboratorio. El diccionario cubre el 100 % del vocabulario de las oraciones evaluadas, de modo que el resultado mide la calidad de las reglas y no la cobertura léxica.

**Figura 13**

*BLEU y chrF de la traducción por tipo de oración*



**Análisis:** el resultado no es uniformemente pobre, y esa es la observación interesante. En oraciones periodísticas literales el sistema alcanza 61,37 de BLEU y 82,28 de chrF, cifras altas porque el lenguaje de agencia es sintácticamente regular y semánticamente literal. En oraciones con entidades y cifras baja a 55,02, y en modismos se desploma a 11,59 de BLEU y 52,46 de chrF. La diferencia entre ambas métricas en esa última categoría es reveladora: chrF, al operar sobre n-gramas de caracteres, sigue premiando el solapamiento parcial de palabras aunque la frase resultante carezca de sentido.

Las ablaciones cuantifican la contribución de cada componente: sin el tratamiento morfológico el BLEU cae de 49,78 a 23,46, sin reordenamiento a 44,90 y sin el diccionario de locuciones a 44,27. La morfología es, con diferencia, el componente más valioso, lo que concuerda con que el español es una lengua flexiva y el inglés no. El intervalo de confianza del 95 % del BLEU global, [34,98 – 62,69], es tan amplio que obliga a tratar la cifra puntual con prudencia: con dieciséis oraciones y una sola traducción de referencia, BLEU es un indicador orientativo.

El fracaso en los modismos es el resultado conceptualmente más claro del apartado. «Quedar en agua de borrajas» no significa nada que pueda componerse a partir del significado de *agua* y *borrajas*: la unidad de traducción no es la palabra sino la expresión completa, y un sistema que opera palabra a palabra no tiene manera de descubrirlo. Como control adicional se entrenó un modelo seq2seq minúsculo con las propias oraciones, que obtiene 85,77 de BLEU sobre las trece de entrenamiento y 2,49 sobre las tres de prueba: una demostración limpia de sobreajuste que ilustra por qué la traducción neuronal exige millones de pares alineados.

#### 4.15 Text generation

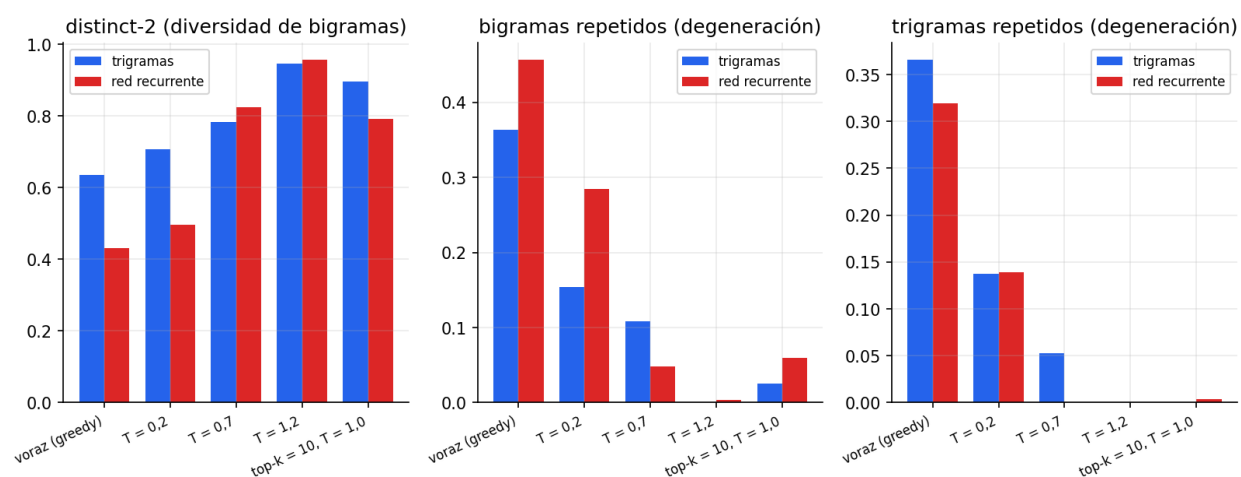
La generación de texto se abordó entrenando modelos desde cero sobre un subconjunto del corpus —1.978 líneas y 57.172 palabras, con el vocabulario recortado a las 5.000 formas más frecuentes, que cubren el 92,6 % de los tokens—. Se compararon modelos de n-gramas con suavizado y una red recurrente *Embedding + Bidirectional LSTM* construida siguiendo el laboratorio de clase (Moroney, 2019).

**Tabla 8***Perplejidad en validación de los modelos de generación*

| Modelo                                                                  | Perplejidad |
|-------------------------------------------------------------------------|-------------|
| Bigramas con suavizado add-k                                            | 407,2       |
| Trigramas con suavizado add-k                                           | 1.199,9     |
| Red <i>Embedding</i> + <i>Bidirectional LSTM</i> (1.408.905 parámetros) | 196,8       |

*Nota.* Menor es mejor. La red alcanzó su mejor punto en la época 7 de 10 con parada temprana, tras 93,7 segundos de entrenamiento en CPU, con una exactitud de predicción de la siguiente palabra del 21,0 %.

El trigrama es **peor** que el bigrama, un resultado que parece paradójico y que la inspección explica: el 73,2 % de los contextos de trigrama observados tiene una única continuación en el corpus. El modelo memoriza el texto de entrenamiento en lugar de aprender regularidades, y ante cualquier contexto nuevo no tiene nada que ofrecer. Es la dispersión de datos en estado puro, y justifica por sí sola el paso a representaciones distribuidas: la red reduce la perplejidad un 51,7 % respecto del bigrama porque generaliza a contextos que nunca vio.

**Figura 14***Efecto de la temperatura y del muestreo sobre la diversidad y la repetición*

**Análisis:** la estrategia de muestreo importa tanto como el modelo. La búsqueda voraz degenera de inmediato: el 45,6 % de sus bigramas y el 32,0 % de sus trigramas son repeticiones, el fenómeno descrito por Holtzman et al. (2020). Con temperatura 0,7 la repetición cae al 0,0 % y la diversidad distinct-2 sube a 0,824; con temperatura 1,2 llega a 0,956 pero el texto pierde coherencia sintáctica. El muestreo top-k con  $k = 10$  ofrece el mejor compromiso observado, con 0,792 de diversidad y solo un 0,4 % de repetición, porque acota la cola de la distribución sin aplanar las probabilidades altas.

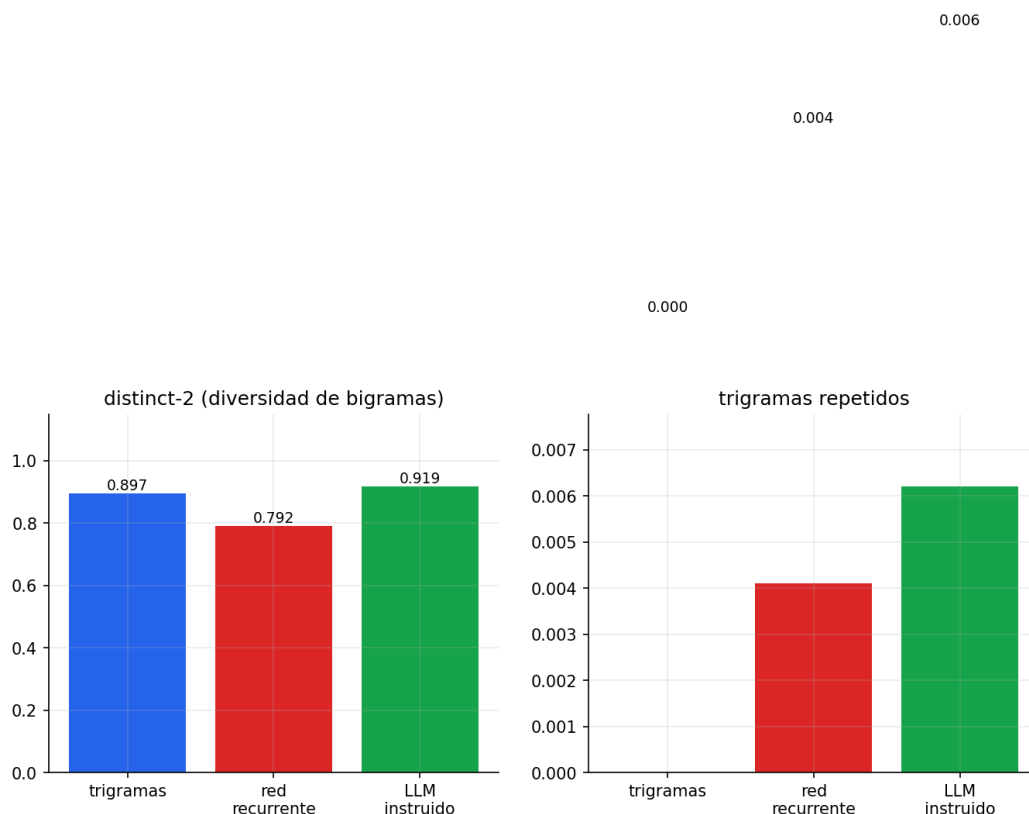
#### 4.16 Generación de texto usando LLM

Este tema es distinto del anterior aunque comparta la mecánica de predecir la siguiente palabra. Un modelo entrenado desde cero aprende de un corpus concreto y solo sirve para el dominio de ese corpus; un gran modelo de lenguaje llega preentrenado sobre volúmenes de texto inalcanzables y se adapta a la tarea mediante instrucciones en el propio texto de entrada, sin actualizar un solo parámetro (Brown et al., 2020). La unidad de trabajo del ingeniero se desplaza del entrenamiento al diseño de la instrucción.

Se registraron diez experimentos de *prompting* sobre noticias del corpus, combinando cuatro tareas —continuación, titulación, resumen en una frase y reescritura de registro— con tres estrategias: *zero-shot*, *few-shot* y cadena de pensamiento. Los resultados se guardaron en `datos/generacion_llm.csv` junto con el prompt exacto que los produjo. **Estas salidas se obtuvieron con el modelo Claude Opus 5 el 27 de septiembre de 2026 en modo conversacional, y no mediante una llamada ejecutada dentro del notebook**, porque el entorno de ejecución no tiene acceso a repositorios de modelos ni a la API; el archivo actúa como registro del experimento y el notebook incluye el código completo, comentado, para reproducirlo con la biblioteca `transformers` o con una API. La declaración explícita de esta diferencia forma parte del resultado: la trazabilidad de cómo se obtuvo un dato es tan importante como el dato.

**Figura 15**

*Comparación entre el modelo entrenado desde cero y el LLM*



**Análisis:** la diferencia de diversidad léxica es grande —distinct-2 de 0,919 frente a 0,824 del mejor ajuste de la red recurrente—, pero el dato revelador es otro: el 16,5 % del léxico que emplea el LLM queda fuera del vocabulario de 5.000 formas con el que se entrenó la red. El modelo pequeño no solo genera peor, sino que literalmente no dispone de las palabras. Ahí se ve el efecto de la escala del preentrenamiento, que no es una mejora incremental de calidad sino un cambio en lo que el modelo puede llegar a decir.

El contraste con el modelo entrenado sobre el corpus deja tres diferencias cualitativas. La primera es el control: al LLM se le pide «resume esto en una sola frase para un teletipo» y obedece, mientras que la red recurrente solo continúa texto, sin noción de tarea. La segunda es el conocimiento del mundo, que permite al LLM situar una noticia de mayo de 2000 en su contexto histórico. La tercera son los riesgos, que son reales y simétricos a las ventajas: la invención de datos verosímiles pero falsos, el costo por llamada, la dependencia de un proveedor externo y, para un trabajo académico, la ausencia de reproducibilidad exacta, ya que la misma instrucción puede producir salidas distintas. Un modelo pequeño entrenado en local es peor y, a la vez, enteramente auditable.



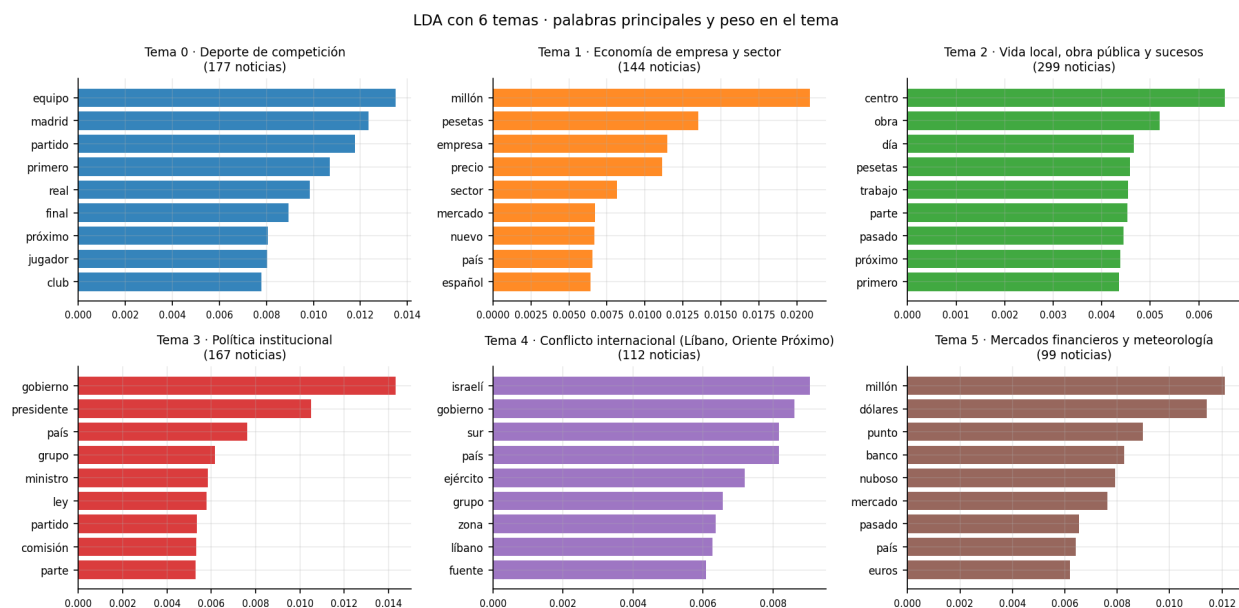
### 4.17 Text mining

La minería de texto se aplicó a las 998 noticias para descubrir estructura sin usar etiquetas. El recuento de n-gramas encabeza con *millón pesetas* (203 apariciones), *millón dólares* (135) y *real madrid* (109), un retrato fiel de la agenda de mayo de 2000. La auditoría del propio proceso detectó que el bigrama *may efe* aparecía 59 veces y ocupaba el sexto puesto antes de filtrar: un residuo de la cabecera de agencia que confirma, ahora desde el otro extremo de la cadena, la necesidad del preprocesamiento del apartado 4.1.

El análisis de colocaciones identificó 259 candidatos y permitió contrastar las dos medidas de asociación. La información mutua puntual privilegia combinaciones raras pero perfectamente fijas —*wall street*, *dow jones*, *cascos azules*—, mientras que la razón de verosimilitud, más estable en frecuencias bajas, devuelve *real madrid*, *medio ambiente* y *sierra leona*. La diferencia entre ambas listas no es un problema de calidad sino de definición: una mide cuán improbable es el encuentro y la otra cuánta evidencia hay de que no sea casual.

**Figura 16**

*Temas descubiertos por asignación latente de Dirichlet*



**Análisis:** con seis temas, el modelo alcanza una coherencia  $c_v$  de 0,467 y una información mutua ajustada de 0,673 frente a las cinco categorías anotadas a mano. El barrido de 5 a 8 temas muestra una tensión instructiva: la coherencia mejora (0,435  $\rightarrow$  0,479) mientras la concordancia con las etiquetas humanas empeora (0,845  $\rightarrow$  0,632). Más temas producen agrupaciones internamente más homogéneas, pero que dejan de corresponder a las categorías que una persona usaría; el número de temas no tiene un óptimo objetivo, sino que depende de para qué se quiera

usar.

El agrupamiento con k-medias sobre la representación latente resultó notablemente más fiel a las categorías humanas que el modelado de temas: con cinco grupos alcanza una información mutua ajustada de **0,890** y una pureza de 0,943, clasificando correctamente 66 de las 70 noticias etiquetadas sin haber visto una sola etiqueta. El coeficiente de silueta, en cambio, es de apenas 0,041, lo que indica que los grupos, pese a ser correctos, no están separados geométricamente: en un espacio de alta dimensión los documentos forman una nube continua y las fronteras entre temas son graduales. Es un recordatorio de que las métricas internas de agrupamiento y las externas pueden discrepar por completo, y de que la elección entre ellas debe justificarse.

## 5. Análisis y discusión

Recorrer diecisiete técnicas sobre un mismo corpus permite algo que no ofrece el estudio aislado de cada una: observar los patrones que se repiten. Esta sección recoge los cinco que atraviesan el laboratorio completo.

### 5.1 Las métricas agregadas ocultan la naturaleza del error

En cuatro temas distintos, la cifra global resultó engañosa y solo la descomposición reveló qué estaba ocurriendo. El etiquetado gramatical marca un 85,5 % que se descompone en 95,2 % fuera de las entidades y 18,5 % dentro, porque lo que se mide ahí es una discrepancia entre convenciones de anotación y no un error del modelo. El reconocimiento de entidades pasa de 0,655 a 0,846 al relajar el tipo, lo que demuestra que el sistema encuentra las entidades y falla al clasificarlas. La traducción promedia 49,78 de BLEU entre un 61,37 en oraciones periodísticas y un 11,59 en modismos, dos regímenes que no tiene sentido promediar. Y el resumen recibe veredictos opuestos según se mida cobertura de entidades, cobertura de términos o redundancia.

La consecuencia metodológica es que una evaluación seria exige decidir de antemano en qué subpoblaciones se va a descomponer el resultado. Sin esa descomposición, las cuatro conclusiones que se habrían extraído en este laboratorio habrían sido erróneas.

## 5.2 Las líneas base sencillas son sorprendentemente difíciles de batir

El laboratorio incluyó deliberadamente una línea base simple en cada tema, y en tres casos esa línea base ganó o empató con el método sofisticado. Un etiquetador que asigna a cada palabra su categoría más frecuente supera a spaCy con el criterio estricto (92,4 % frente a 85,5 %), y solo se derrumba en las palabras desconocidas (32,3 % frente a 65,6 %), que es exactamente donde se paga el precio de memorizar en lugar de generalizar. En el resumen, Lead-3 supera a TextRank en cobertura de entidades y en redundancia, hasta el punto de que TextRank converge hacia la entradilla en 11 de 12 noticias. En la clasificación, TF-IDF con regresión logística alcanza 0,957 frente al 1,000 de los embeddings, una diferencia de dos noticias sobre setenta.

El valor de la línea base no es competitivo sino epistemológico: obliga a precisar qué mejora exactamente el método complejo y en qué condiciones. Sin ella, cualquier resultado razonable parece un éxito.

## 5.3 Hay un límite estructural común a todas las representaciones de bolsa de palabras

El hallazgo más transversal del laboratorio es que cinco representaciones muy distintas —TF-IDF, análisis semántico latente, vectores de spaCy, Word2Vec y su versión ponderada— asignan **exactamente 1,00** de similitud a «el Real Madrid venció al Valencia» y «el Valencia venció al Real Madrid», donde la anotación humana da 0,33. No es un defecto de entrenamiento que más datos pudieran corregir: todas componen la oración promediando vectores de palabra, y un promedio es invariante ante el orden. La negación falla por una razón emparentada, ya que el efecto semántico de *no* es invertir el valor de verdad de la oración completa, algo que ninguna suma ponderada puede representar.

Este límite explica hechos dispersos por todo el informe: por qué las listas de stopwords eliminan las negaciones sin que a nadie le parezca grave en clasificación temática, por qué el resumen extractivo no puede reformular y por qué las preguntas de causa obtienen 0,0 de F1. Y delimita con precisión qué aporta la arquitectura Transformer (Vaswani et al., 2017): no vectores mejores, sino una representación de cada token construida en función de los demás, es decir, sensible al orden y a la estructura. Los resultados de este laboratorio permiten afirmar dónde está el techo del enfoque clásico y por qué ese techo existe, aunque no permitan medir empíricamente cuánto lo supera un Transformer.

#### 5.4 Las representaciones densas se pagan solas únicamente en el régimen de pocos datos

La curva de aprendizaje de la clasificación es el resultado cuantitativo más accionable del laboratorio: con un solo ejemplo etiquetado por tema, los embeddings superan a TF-IDF en 22,6 puntos de exactitud; con diez ejemplos, la diferencia se reduce a 1,8 puntos. La causa es la brecha léxica medida en el apartado 4.10: donde la anotación humana ve 0,97 de similitud entre dos frases que significan lo mismo sin compartir palabras, TF-IDF ve 0,045 y los embeddings entre 0,77 y 0,79.

De ahí se sigue un criterio de ingeniería directo: cuando etiquetar es caro y los ejemplos escasean, la representación densa es la inversión rentable; cuando hay datos etiquetados abundantes y el vocabulario del dominio es estable, un modelo léxico bien normalizado es más barato, más rápido y más fácil de depurar, porque sus coeficientes son directamente interpretables.

#### 5.5 Contar frente a generalizar: el mismo problema en dos escalas

Dos resultados aparentemente inconexos son la misma observación. En el modelo de n-gramas, el trigramma es peor que el bigrama (1.199,9 frente a 407,2 de perplejidad) porque el 73,2 % de los contextos de trigramma tiene una única continuación en el corpus: el modelo memoriza el texto en vez de aprender regularidades. En la representación de documentos, TF-IDF asigna similitud cero a dos textos del mismo tema que no comparten palabras. En ambos casos el sistema cuenta coincidencias exactas y, en cuanto el fenómeno observado no aparece literalmente en los datos, no tiene nada que ofrecer.

La respuesta también es la misma en las dos escalas: sustituir el conteo por una representación distribuida que sitúe cerca lo que se comporta parecido. La red recurrente reduce la perplejidad un 51,7 % respecto del bigrama, y los embeddings resuelven la brecha léxica. Es útil reconocer que la historia del área, desde el modelo vectorial hasta los grandes modelos de lenguaje, es en buena medida la aplicación reiterada de esa misma idea a escalas crecientes.

### 6. Limitaciones

Las limitaciones del trabajo condicionan el alcance de sus conclusiones y conviene enunciarlas con precisión.

- **Tamaño de los conjuntos de evaluación.** Los cinco conjuntos anotados suman 191 elementos (42 lemas, 26 pares de similitud, 70 noticias, 37 preguntas y 16 oraciones). Bastan para comparar métodos y analizar errores concretos, pero no para estimar el rendimiento absoluto con precisión: el intervalo de confianza del 95 % del BLEU de traducción, [34,98 – 62,69], lo ilustra con claridad.

- **Anotador único.** Los conjuntos fueron anotados por el autor, sin un segundo anotador ni medida de acuerdo entre anotadores. En tareas como la similitud semántica, donde el juicio humano es intrínsecamente variable, esto introduce un sesgo que no puede cuantificarse con los datos disponibles.
- **Un solo género y una sola época.** El corpus son despachos de agencia de mayo de 2000. Su lenguaje es sintácticamente regular y literal, lo que favorece a los métodos basados en reglas y explica en parte el buen desempeño de Lead-3 y del traductor. Además, el vocabulario está fechado —las cifras están en pesetas— y las entidades corresponden a un momento histórico concreto. Las conclusiones no se trasladan sin más a redes sociales, conversación o texto técnico.
- **Ausencia de comparación empírica con modelos Transformer.** El entorno de ejecución no tenía acceso al repositorio de modelos preentrenados, de modo que las afirmaciones sobre lo que aportaría un codificador contextual se apoyan en la bibliografía y en el análisis de los errores observados, no en mediciones propias. Los notebooks incluyen el código correspondiente para ejecutarlo en un entorno con acceso, lo que permitiría cerrar esa comparación.
- **Recursos de cómputo.** Todo se ejecutó en CPU. La red recurrente de generación se entrenó durante 94 segundos sobre el 15 % del corpus con un vocabulario de 5.000 formas; sus resultados ilustran el comportamiento cualitativo de la arquitectura, no su capacidad real.
- **Decisiones de evaluación que afectan a las cifras.** El mapeo entre las etiquetas EAGLES del corpus y las categorías universales de spaCy es una decisión propia y documentada que condiciona la exactitud reportada en el apartado 4.7; el uso de Lead-3 como pseudo-referencia para ROUGE en el apartado 4.13 no sustituye a resúmenes humanos.

## 7. Conclusiones

- El preprocesamiento no es un paso previo y mecánico sino una cadena de decisiones medibles. La tokenización reduce el vocabulario un 29,5 %, más que cualquier otra operación; el paso a minúsculas funde una entidad con una palabra común en 613 formas que concentran el 21,6 % de los tokens de entidad; y quitar las tildes daña más de cuatro tokens por cada uno que corrige. Cada una de esas decisiones se tomó tras medir su costo, no por convención.
- El orden de las operaciones determina el techo de rendimiento alcanzable. El reconocimiento de entidades debe ejecutarse sobre el texto original y antes de la normalización agresiva; lo contrario impone una pérdida que ningún modelo posterior

puede recuperar.

- Las herramientas heredadas de otro idioma fallan de forma silenciosa. El *Tokenizer* del laboratorio de clase no filtra los signos de apertura del español, y su lista de stopwords en inglés elimina del texto en español el número *once*. En ambos casos el código se ejecuta sin errores y el resultado parece razonable.
- En la ponderación de términos, la normalización de longitud aporta más que el propio IDF (+0,126 frente a +0,037 de precisión@5), un orden de importancia inverso al de la exposición habitual del tema. El IDF, verificado contra scikit-learn con diferencia máxima de 0,0, cambia el término más representativo en el 51,9 % de las noticias.
- Las representaciones densas resuelven la brecha léxica y se pagan solas cuando hay pocos ejemplos etiquetados (22,6 puntos de ventaja con un ejemplo por clase, 1,8 con diez), pero comparten con las dispersas un límite estructural: al componer por promedio son ciegas al orden y a la negación, y asignan similitud 1,00 a dos oraciones con los papeles invertidos.
- Los sistemas clásicos contruidos para este laboratorio alcanzan resultados útiles y, sobre todo, delimitan con precisión dónde está su techo: la recuperación en preguntas y respuestas es perfecta a cinco oraciones y el fallo está en la extracción del tramo; la traducción por reglas funciona en prosa literal (61,37 de BLEU) y se desploma en modismos (11,59); el resumen extractivo no puede reformular. Cada uno de esos límites nombra exactamente la función que cumple un modelo preentrenado.
- En generación, la estrategia de muestreo pesa tanto como el modelo: la búsqueda voraz degenera con un 45,6 % de bigramas repetidos, mientras que el muestreo con temperatura 0,7 o top-k con  $k = 10$  elimina prácticamente la repetición. Y el salto del conteo a la representación distribuida reduce la perplejidad un 51,7 %, la misma idea que resuelve la brecha léxica una escala más abajo.
- La trazabilidad forma parte del resultado. Las salidas obtenidas con un gran modelo de lenguaje se documentan como lo que son —registro de una interacción conversacional, no una llamada ejecutada en el notebook—, y cada cifra de este informe puede rastrearse hasta el archivo de métricas del notebook que la produjo.

## 8. Referencias

- Arora, S., Liang, Y. y Ma, T. (2017). A simple but tough-to-beat baseline for sentence embeddings. En *Proceedings of the 5th International Conference on Learning Representations*.
- Bird, S., Klein, E. y Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.

- Blei, D. M., Ng, A. Y. y Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research*, 3, 993–1022.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Church, K. W. y Hanks, P. (1990). Word association norms, mutual information, and lexicography. *Computational Linguistics*, 16(1), 22–29.
- Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K. y Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6), 391–407.
- Devlin, J., Chang, M.-W., Lee, K. y Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. En *Proceedings of NAACL-HLT 2019* (pp. 4171–4186).
- Erkan, G. y Radev, D. R. (2004). LexRank: Graph-based lexical centrality as salience in text summarization. *Journal of Artificial Intelligence Research*, 22, 457–479.
- Gómez Castañeda, Y. N. (2026). *lab-embeddings: Transforma texto en embeddings* [Repositorio]. GitHub. <https://github.com/Nicolas2322/lab-embeddings>
- Holtzman, A., Buys, J., Du, L., Forbes, M. y Choi, Y. (2020). The curious case of neural text degeneration. En *Proceedings of the 8th International Conference on Learning Representations*.
- Honnibal, M., Montani, I., Van Landeghem, S. y Boyd, A. (2020). *spaCy: Industrial-strength natural language processing in Python*. Zenodo. <https://doi.org/10.5281/zenodo.1212303>
- Jurafsky, D. y Martin, J. H. (2025). *Speech and language processing* (3.ª ed., borrador). Stanford University.
- Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. En *Text Summarization Branches Out* (pp. 74–81). Association for Computational Linguistics.
- Mihalcea, R. y Tarau, P. (2004). TextRank: Bringing order into texts. En *Proceedings of EMNLP 2004* (pp. 404–411).
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. y Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 3111–3119.
- Moroney, L. (2019). *dlaicourse: Notebooks for learning deep learning* [Repositorio]. GitHub. <https://github.com/lmoroney/dlaicourse>
- Nivre, J., de Marneffe, M.-C., Ginter, F., Hajič, J., Manning, C. D., Pyysalo, S., Schuster, S., Tyers, F. y Zeman, D. (2020). Universal Dependencies v2: An evergrowing multilingual treebank collection. En *Proceedings of LREC 2020* (pp. 4034–4043).

- Papineni, K., Roukos, S., Ward, T. y Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. En *Proceedings of ACL 2002* (pp. 311–318).
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Popović, M. (2015). chrF: Character n-gram F-score for automatic MT evaluation. En *Proceedings of the Tenth Workshop on Statistical Machine Translation* (pp. 392–395).
- Porter, M. F. (2001). *Snowball: A language for stemming algorithms*.  
<https://snowballstem.org/texts/introduction.html>
- Rajpurkar, P., Jia, R. y Liang, P. (2018). Know what you don't know: Unanswerable questions for SQuAD. En *Proceedings of ACL 2018* (pp. 784–789).
- Reimers, N. y Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. En *Proceedings of EMNLP-IJCNLP 2019* (pp. 3982–3992).
- Robertson, S. y Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Salton, G. y Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
- Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.
- Tjong Kim Sang, E. F. (2002). Introduction to the CoNLL-2002 shared task: Language-independent named entity recognition. En *Proceedings of CoNLL-2002* (pp. 155–158).
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. y Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- Řehůřek, R. y Sojka, P. (2010). Software framework for topic modelling with large corpora. En *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks* (pp. 45–50).